

Open-Source Actions – Unified CI Dispatcher

(README)

Open-Source Actions is a PowerShell-based toolkit that unifies various LabVIEW build and CI tasks under a single dispatcher. This project provides a **unified endpoint** (`Invoke-OSAction.ps1`) that routes a high-level **ActionName** to a specific adapter script, standardizing how CI tasks are invoked ¹ ². It enables both GitHub Actions workflows and local PowerShell users to call any supported LabVIEW CI action through a common interface, improving consistency and reducing duplicated script logic.

Quickstart

1. **Install Requirements:** Ensure you have **NI LabVIEW** (with command-line interface support, often via *g-cli*) installed on the target runner. Most actions require LabVIEW and the NI *g-cli* tool to be available (Windows runners are recommended ³). Also verify PowerShell 7+ (`pwsh`) is available for cross-platform script execution.
2. **Invoke via Composite Action (GitHub):** Use the provided composite action in your workflow. For example, to **build a LabVIEW Packed Library** using this unified dispatcher:

```
jobs:
  build_lvlibp:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v3
      - name: Build Packed Library (32-bit)
        uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
        with:
          action_name: build-lvlibp
          args_json: >
            {
              "MinimumSupportedLVVersion": "2019",
              "SupportedBitness": "32",
              "RelativePath": ".",
              "LabVIEW_Project": "MyProject.lvproj",
              "Build_Spec": "My Build",
              "Major": 1, "Minor": 0, "Patch": 0,
              "Build": 123, "Commit": "abcdef"
            }
            }
```

In this step, the composite action invokes the dispatcher to run the **Build** task. The `args_json` contains all parameters the action needs (here, to build a 32-bit LV library). The dispatcher will locate the appropriate script and execute it, failing the step if a problem occurs (non-zero exit).

1. **Invoke via PowerShell (CLI):** You can also call the dispatcher script directly. For example, the above build can be run in a PowerShell session or script:

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName build-lvlibp -ArgsJson '{
  "MinimumSupportedLVVersion": "2019",
  "SupportedBitness": "32",
  "RelativePath": ".",
  "LabVIEW_Project": "MyProject.lvproj",
  "Build_Spec": "My Build",
  "Major": 1, "Minor": 0, "Patch": 0,
  "Build": 123, "Commit": "abcdef"
}'
```

This will import the **OpenSourceActions** module and run the **Build** adapter. On completion, the script returns with an exit code (0 for success or a non-zero error code). You can include optional flags like `-WorkingDirectory` to change directory before execution, or `-DryRun` to simulate the action (see below).

1. **Confirm Results:** After running, check the console output and exit code. The unified dispatcher prints informative logs (at INFO level by default) and any errors encountered. For GitHub Actions, a non-zero exit code will mark the step as failed, surfacing any error messages thrown by the adapter or underlying script ⁴ ⁵.

How It Works (Unified Dispatcher Usage)

The heart of the system is `Invoke-OSAction.ps1`, a dispatcher script that takes two primary inputs: an **ActionName** (identifying which action to run) and an **ArgsJson** string (providing the arguments for that action) ⁶ ⁷. The dispatcher loads the **OpenSourceActions** PowerShell module, which contains *adapter functions* for each action. It then:

- **Parses ArgsJson:** The JSON string is parsed into a PowerShell hashtable of arguments. Missing or malformed JSON triggers an error. Each adapter expects specific keys in the JSON – unknown or missing required keys will result in a *terminating error* before any action is run ⁷ ⁸.
- **Selects the Action Adapter:** A registry maps the provided ActionName (case-insensitive) to the corresponding adapter function (e.g. `"apply-vipc"` maps to `InvokeApplyVIPC`) ⁹ ¹⁰. If an unknown ActionName is given, the dispatcher halts and lists the allowed names in the error message ¹¹.
- **Sets Up Environment:** If a `-WorkingDirectory` is specified, the dispatcher will `Push-Location` to that folder so that relative file paths in the action's logic resolve correctly ¹². After execution, it returns to the original directory.
- **Invokes the Adapter:** The dispatcher calls the adapter function corresponding to the action. The adapter receives the parsed arguments (plus uniform parameters like `LogLevel` and `DryRun`) and in turn calls the underlying script that performs the actual work ¹³ ¹⁴. All adapter functions are defined with `[CmdletBinding()]` and strict error checking (`Set-StrictMode -Version`

`Latest` and `$ErrorActionPreference = 'Stop'`), so any error in execution will throw an exception ¹⁵.

- **Handles Results:** After the adapter (and thus the underlying script) finishes, the dispatcher captures the script's `$LASTEXITCODE`. A zero exit code is treated as success. If a non-zero exit occurs or an exception was thrown, the dispatcher will treat it as a failure: it writes an error message (including the underlying script's output or thrown message) and ensures a non-zero exit code is returned ¹⁶ ¹⁷. The exit code is **normalized** – for example, if an underlying script signals “partial success” with a specific code (like 2 or 3 for test failures), that code is preserved rather than forcing it to 1 ⁵. This allows workflows to distinguish types of failures if needed.

Logging: Logging verbosity is controlled by the `-LogLevel` parameter. By default, `INFO` level logs are shown, which include key steps of the action. Set `LogLevel` to `DEBUG` for more detailed internal logs (which enables PowerShell verbose output) ¹⁸ ¹⁹. The dispatcher prints a header indicating which action is being run, the working directory, and whether `DryRun` is active ²⁰. Adapters use `Write-Information` for high-level info and `Write-Verbose` for debug details, and `Write-Error` for any fatal issues ²¹.

Discovery Features

To assist users, the unified dispatcher supports discovery commands to enumerate available actions and their inputs:

- **List Actions:** Running `Invoke-OSAction.ps1 -ListActions` (with no `ActionName`) will output a list of all supported `ActionName` values and a brief description of each action's purpose. This helps identify what actions are available in the module (e.g. `add-token-to-labview`, `build-lvlibp`, etc.) without reading the source. Use this to discover the exact `ActionName` spelling and casing.
- **Describe Action:** You can get detailed input information for a specific action by running `Invoke-OSAction.ps1 -Describe <ActionName>`. This will print the expected parameters (required and optional) for that action, along with short descriptions. It's essentially an in-tool help for the adapter. For example, `Invoke-OSAction.ps1 -Describe run-unit-tests` might show that it requires **MinimumSupportedLVVersion** and **SupportedBitness** parameters and explain their meaning.

Note: The `-ListActions` and `-Describe` flags are mutually exclusive with `-ActionName`. If any of these discovery flags are present, the dispatcher will not execute an actual action. They are provided to improve usability and reduce mistakes in usage ²². (Behind the scenes, the module may generate this info from the registry or from each adapter's parameter metadata.)

Available Actions

The following actions are supported by the `OpenSourceActions` module (`ActionName` in parentheses). Click each for detailed documentation on inputs and usage:

- **Add Token to LabVIEW** (`add-token-to-labview`) – Add tokens to LabVIEW configuration INI (for enabling hidden features) ²³.
- **Apply VIPC Package** (`apply-vipc`) – Apply a VI Package Configuration (.vipc) to a LabVIEW installation ²⁴.

- **Build Packed Library** (`build-lvlibp`) – Build a LabVIEW Packed Project Library (.lvlibp) from a project and build spec ²⁵.
- **Build VI Package** (`build-vi-package`) – Update VIPB metadata and build a VI Package (.vip) using a .vipb build spec ²⁶.
- **Build (Composite)** (`build`) – Orchestrate a full build: build 32- & 64-bit libraries, update metadata, and produce a package ²⁷.
- **Close LabVIEW** (`close-labview`) – Gracefully quit the LabVIEW IDE if running, given version and bitness ²⁸.
- **Generate Release Notes** (`generate-release-notes`) – Create a markdown release notes file from Git commit history ²⁹.
- **Missing in Project** (`missing-in-project`) – Check a LabVIEW project for missing items (reports if VIs are missing) ³⁰.
- **Modify VIPB Display Info** (`modify-vipb-display-info`) – Modify product name, version, etc., in a VIPB file's display information (for packages) ³¹.
- **Prepare LabVIEW Source** (`prepare-labview-source`) – Prepare source code for packaging by running a build spec to zip source and remove dev-only tokens ³².
- **Rename File** (`rename-file`) – Rename a file on disk (with error handling to avoid collisions) ³³.
- **Restore LV Source** (`restore-setup-lv-source`) – Restore a packaged LabVIEW source (unzip it and remove dev tokens), effectively undoing a prepare step ³⁴.
- **Revert Development Mode** (`revert-development-mode`) – Convert a repository from “development mode” (source code form) back to a packaged state, for both 32-bit and 64-bit, and close LabVIEW ³⁵.
- **Run Unit Tests** (`run-unit-tests`) – Execute LabVIEW unit tests in a project, produce a test report, and output pass/fail summary (with special exit codes) ³⁶.
- **Set Development Mode** (`set-development-mode`) – Prepare a repository for development work: removes built artifacts, adds dev tokens, extracts sources, etc., for both 32 & 64-bit ³⁷.

Each action's documentation (see **docs/actions/** directory) provides details on its inputs, examples, and specific behaviors.

Cross-Platform and Runner Guidance

Most actions are **designed for Windows runners** because they rely on LabVIEW and the NI g-cli (LabVIEW's command-line interface) which are typically available on Windows only ³ ³⁸. Key considerations:

- **Windows / LabVIEW:** Use a Windows GitHub runner (e.g., `windows-latest`) for any action that interacts with LabVIEW. Ensure the appropriate LabVIEW version(s) are installed, along with any required toolkits or VI Packages for your project. The actions accept a **MinimumSupportedLVVersion** and **SupportedBitness** to target specific LabVIEW installations.
- **Linux / macOS:** Pure file or Git operations (like *Generate Release Notes* or *Rename File*) can run on PowerShell Core on Linux/macOS. However, these actions should be used with caution on non-Windows platforms. The module attempts to detect unsupported scenarios (e.g., trying to call LabVIEW on Linux) and will throw an error early if the environment is not suitable ³⁹. In general, if an action involves g-cli or LabVIEW, **it will not function on Linux/macOS runners**.
- **PowerShell Core 7+:** The dispatcher and adapters are compatible with PowerShell 7 (Core), which is used in GitHub's `pwsh` shell. They enforce strict mode and are tested under PowerShell 7. If

running on older Windows PowerShell (5.1), functionality should remain, but PowerShell Core is recommended for consistency across environments ⁴⁰ .

- **g-cli in PATH:** Ensure that the LabVIEW CLI (`labviewcli` or relevant executable) is accessible. The scripts typically call `g-cli` which should be configured to point to the correct LabVIEW command-line interface. You may need to install NI's LabVIEW CLI tool or ensure environment variables are set so the scripts find LabVIEW.

Example Workflow

Below is an example workflow leveraging multiple Open-Source Actions in sequence to illustrate a typical CI/CD pipeline for a LabVIEW project:

```
name: LabVIEW CI
on: [push]
jobs:
  build_test_package:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set Development Mode (Prepare Source)
        uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
        with:
          action_name: set-development-mode
          args_json: '{"RelativePath": "."}'

      - name: Run Unit Tests
        uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
        with:
          action_name: run-unit-tests
          args_json: '{"MinimumSupportedLVVersion": "2020", "SupportedBitness":
"64"}'

      - name: Build Library and Package
        uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
        with:
          action_name: build
          args_json: >
            {
              "RelativePath": ".",
              "Major": 1, "Minor": 2, "Patch": 3,
              "Build": 0, "Commit": "${{ github.sha }}",
              "LabVIEWMinorRevision": "f1",
              "CompanyName": "YourOrg",
              "AuthorName": "YourName"
            }
```

```
- name: Revert from Development Mode
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: revert-development-mode
    args_json: '{"RelativePath": "."}'
```

What this does: The workflow checks out code, sets the repo to development mode (unpacking sources and adding tokens), runs unit tests, then builds the project into a packaged library and VI Package (using the combined **build** orchestrator), and finally reverts the repo out of development mode (cleaning up tokens and temporary files). Each step uses the unified `abstract-action` with a specific `action_name` and JSON arguments. If any step fails (returns a non-zero exit code), the job will stop and be marked as failed.

SemVer Versioning

This project follows **Semantic Versioning (SemVer)** for the **OpenSourceActions** PowerShell module and composite action. The version is defined in the module manifest (`OpenSourceActions.psd1`) and reflected in release tags:

- **MAJOR** version increments for breaking changes: Removing or renaming an action, changing required parameter names/types, or any update that could break existing workflows. (E.g., dropping support for an older LabVIEW version in a way that changes behavior might warrant a major bump.)
- **MINOR** version increments for new features and non-breaking improvements: Adding a new action adapter, introducing optional parameters to existing actions (while maintaining backward compatibility), or enhancements to logging and DryRun behavior. Users can upgrade minor versions without changing their workflows, gaining new functionality.
- **PATCH** version increments for bug fixes and documentation updates: Internal fixes that do not affect the interface or outcomes, such as fixing a script's logic, adjusting default values without breaking expected behavior, or improving documentation and comments.

For example, going from `v1.2.0` to `v1.3.0` indicates a new action or feature was added, whereas `v2.0.0` would signal a significant change like an incompatible interface update. The module's version is updated in **OpenSourceActions.psd1** (the `ModuleVersion` field) accordingly, and each release is accompanied by **Release Notes**.

Please refer to **docs/versioning.md** for details on how to update versions and release notes when contributing changes. In general, any time the module version changes, update the manifest and include a section in the release notes describing what changed (especially for major or minor updates).

Getting Help

- **Documentation:** See the markdown files in the `docs/` directory for detailed guides:
- **Actions Reference:** `docs/actions/` (one file per action) – input descriptions, usage examples, and error codes.
- **Developer Guide:** `docs/developer-guide.md` – how to extend this toolkit with new actions.

- **Design Rationale:** docs/design.md – the architectural reasoning behind this unified dispatcher.
- **Versioning Policy:** docs/versioning.md – maintaining semantic versioning for this module.
- **Migration Guide:** docs/migration.md – advice for migrating existing pipelines to use the unified dispatcher.
- **List Available Actions:** Run `Invoke-OSAction.ps1 -ListActions` to see all action names.
- **Show Action Inputs:** Run `Invoke-OSAction.ps1 -Describe <ActionName>` for usage info.
- **GitHub Issues:** For any bugs or feature requests, please open an issue in the repository. Include relevant logs (run with `-LogLevel DEBUG` for more info) and your LabVIEW version details.

Add Token to LabVIEW (Action Reference)

Action Name: `add-token-to-labview`

Purpose

The **Add Token to LabVIEW** action adds one or more *tokens* to the LabVIEW configuration INI file for a given LabVIEW installation. This is typically used to enable certain LabVIEW environment settings or experimental features by adding keys to LabVIEW's INI config. Internally, this action calls a LabVIEW VI (via the g-cli interface) that performs the token addition ²³. It also resolves the provided relative path to an absolute path (for the project or INI location) and ensures the LabVIEW version specified is supported ²³.

Inputs

Required:

- **MinimumSupportedLVVersion** (string): The minimum LabVIEW version that must be installed to perform this action. This is used to select the LabVIEW instance (especially if multiple versions are present) and to validate compatibility.
- **SupportedBitness** (string): The CPU bitness ("32" or "64") of the LabVIEW version to target. This determines whether to use 32-bit or 64-bit LabVIEW for adding the token.
- **RelativePath** (string): A path (relative to the repository or working directory) that may influence where the LabVIEW INI is located or which project's context is used. Typically this would be the path to the LabVIEW project or directory whose LabVIEW instance's INI should be modified.

Optional: None. (*All inputs are mandatory for this action.*)

In addition, this action supports the common **LogLevel** and **DryRun** options via the unified dispatcher (see notes below).

Usage (Command Line)

To call this action via PowerShell, run the dispatcher with `ActionName` **add-token-to-labview** and provide the required parameters in JSON. For example:

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName add-token-to-labview -
ArgsJson '{
  "MinimumSupportedLVVersion": "2020",
  "SupportedBitness": "64",
  "RelativePath": "MyProjectFolder"
}'
```

This will invoke the AddTokenToLabVIEW adapter, which in turn runs the underlying AddTokenToLabVIEW.ps1 script with the specified parameters. On success, the LabVIEW INI file for the targeted LabVIEW version will have the new tokens added.

Usage (GitHub Actions)

In a GitHub Actions workflow (YAML), use the composite action wrapper:

```
- name: Add tokens to LabVIEW INI
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: add-token-to-labview
    args_json: '{"MinimumSupportedLVVersion": "2020", "SupportedBitness": "64",
"RelativePath": "."}'
```

This step will add the tokens as specified. Typically, you run this to prepare the LabVIEW environment (for example, enabling VI Server or hidden features required for later build steps). It should be run on a **Windows runner** that has the specified LabVIEW version installed.

Outputs and Exit Codes

This action does not produce structured output (no return data besides logs). It writes informational logs about the token addition process and prints the command/result from the g-cli call ⁴¹.

- **Exit Code 0:** Tokens were successfully added to the LabVIEW INI. The step will be marked success.
- **Non-Zero Exit Code:** An error occurred. The underlying script or LabVIEW CLI likely encountered an issue (e.g., LabVIEW not found, permission error writing INI). The adapter will catch this and throw; the dispatcher will log the error and return a non-zero exit code (causing the GitHub step to fail) ⁴². Specifically, the script sets `$LASTEXITCODE` non-zero on failure and propagates that to the caller

⁴².

Dry Run Behavior

If **DryRun** is enabled for this action, the adapter will **simulate** adding the token without actually calling the LabVIEW CLI. It will log a message indicating what command *would* be executed, including the target INI path and token parameters, and then exit without altering any files. In DryRun mode, the exit code will be

0 as long as the inputs are valid (since no real error can occur if not executing the CLI). Use DryRun to test your configuration in CI without making changes to the LabVIEW environment.

Notes

- **LabVIEW & g-cli:** This action requires NI LabVIEW (at least the version specified in *MinimumSupportedLVVersion*) to be installed, along with the LabVIEW CLI (*g-cli*) tool. It will attempt to open LabVIEW (headlessly) to add the token. Ensure the runner has the proper version and bitness of LabVIEW available.
 - **Persistent Side Effect:** The added tokens persist in the LabVIEW configuration INI on the machine. Once added, they remain for that LabVIEW installation until removed. This means if you run this on a self-hosted runner or a persistent machine, the change is permanent (which might be desired for enabling a setting).
 - **Idempotency:** Adding the same token multiple times typically has no additional effect (LabVIEW will ignore duplicate tokens in the INI). Thus, running this action multiple times with the same inputs is generally harmless aside from repeated log messages. However, avoid adding tokens unnecessarily on every run if not needed.
 - **Error Handling:** If the LabVIEW CLI VI fails (for example, if the token is unrecognized or LabVIEW isn't available), the script will throw an error. The unified dispatcher will catch this and treat it as a failure with an appropriate message.
-

Apply VIPC Package (Action Reference)

Action Name: `apply-vipc`

Purpose

Apply VIPC applies a VI Package Configuration (.vipc) file to a LabVIEW installation. A VIPC file typically contains a set of VI packages and perhaps configuration data to deploy to LabVIEW (using VI Package Manager infrastructure). This action ensures the correct LabVIEW version is targeted and then calls the LabVIEW CLI to apply the VIPC ²⁴. It handles version string conversion (some versions might be specified as enums or aliases and are converted to numeric strings) and locates the .vipc file on disk before invoking the apply. In short, this automates installing a batch of VI Packages into the specified LabVIEW environment.

Inputs

Required:

- **MinimumSupportedLVVersion** (`string`): The minimum LabVIEW version required (and to use) for applying the VIPC. This helps pick the correct LabVIEW instance.
- **VIP_LVVersion** (`string`): The LabVIEW version for which the VIPC is intended (e.g., `"2019"` or `"2020SP1"`). This may be used internally by the VIPC apply process to ensure compatibility.

- **SupportedBitness** (string): "32" or "64", indicating the LabVIEW bitness to use when applying the package.
- **RelativePath** (string): A relative path that helps locate the .vipc file or the project context. Usually, the .vipc path might be relative to the repository, so this ensures the working directory context.
- **VIPCPATH** (string): The path to the .vipc file to apply. This can be relative (if so, it's resolved against the RelativePath or working directory) or absolute. This file contains the packages to install.

Optional: None (all above must be provided for a valid invocation).

(As with other actions, LogLevel and DryRun are supported via the dispatcher.)

Usage (Command Line)

Invoke via PowerShell:

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName apply-vipc -ArgsJson '{
  "MinimumSupportedLVVersion": "2019",
  "VIP_LVVersion": "2019",
  "SupportedBitness": "64",
  "RelativePath": ".",
  "VIPCPATH": "MyProject.vipc"
}'
```

This will trigger the ApplyVIPC adapter which runs `ApplyVIPC.ps1` under the hood. The script will find "MyProject.vipc" in the current directory, ensure LabVIEW 2019 (64-bit) is available, and then call the g-cli to apply the package. Any output from the package application (like progress or errors from VI Package Manager) will be printed to the console.

Usage (GitHub Actions)

Example usage in a workflow:

```
- name: Apply VIPC Package
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: apply-vipc
    args_json: '{"MinimumSupportedLVVersion": "2019", "VIP_LVVersion": "2019",
  "SupportedBitness": "64", "RelativePath": ".", "VIPCPATH": "MyProject.vipc"}'
```

Ensure that the VIPC file (in this case `MyProject.vipc`) is present on the runner (for instance, if it's in your repo, make sure to check out the code before this step). The step will run the VIPC application on the specified LabVIEW version.

Outputs and Exit Codes

- **Exit Code 0:** The VIPC was successfully applied with no errors ⁴³. All packages in the VIPC are installed to the LabVIEW environment.
- **Non-Zero Exit Code:** An error occurred during application. The underlying script prints an error message and throws if the apply failed ⁴³. Possible causes include: the .vipc file not found, incompatible LabVIEW version, or package installation failures. The adapter will catch the error and propagate a failure. Notably, the script distinguishes error conditions by throwing; any thrown error will result in the dispatcher returning exit code 1 (or a specific code if thrown as such).

The action logs the output of the g-cli VIPC apply process to the console ⁴⁴. On success, you might see messages about packages applied. On failure, you will see the error message from the CLI or the script (which could include which package failed or if LabVIEW couldn't be launched).

Dry Run Behavior

With **DryRun** enabled, the Apply VIPC action will **not actually apply** the package. Instead, it will log what it *would* do – e.g., “DryRun: would apply VIPC at `<path>` to LabVIEW `<version>` (`<bitness>`)” – and then exit without calling LabVIEW. This can be useful to verify that the .vipc file path and parameters are correct. In DryRun, the exit code will be 0 as long as the parameters are valid (DryRun skips the actual apply which is where errors would normally occur).

Notes

- **LabVIEW & VIPM:** Applying a VIPC requires that the VI Package Manager (VIPM) or LabVIEW's package API is available. The g-cli will handle this if LabVIEW is installed. Ensure the target LabVIEW version has VIPM or support for applying packages.
- **Platform:** This action must run on Windows with LabVIEW installed. It performs file operations (accessing the .vipc) and uses LabVIEW's CLI – it will not run on Linux/macOS.
- **Multiple Applications:** Generally, a VIPC is applied once to set up the environment. Re-applying the same VIPC to the same LabVIEW might skip already installed packages or re-install them if allowed. The script itself does conversions of version strings and ensures the environment is correct each time. It modifies the LabVIEW installation by installing components (side effect on the LabVIEW env).
- **Error Handling:** If a specific package in the VIPC fails to install, the action as a whole is considered failed. You may need to troubleshoot by opening LabVIEW or VIPM logs if this fails in CI. The unified log will capture the high-level error, but details might be found in LabVIEW's CLI output.

Build LabVIEW Packed Library (Action Reference)

Action Name: `build-lvlibp`

Purpose

Build LabVIEW Packed Library compiles a LabVIEW project's build specification into a **Packed Project Library (.lvlibp)**. A packed library is a deployable, versioned library file. This action uses LabVIEW's CLI to run a "Build Packed Library" operation on a given project and build spec. It automates injecting version information (Major/Minor/Patch/Build/Commit) into the build, ensuring output directories are prepared, and calling LabVIEW to perform the build ²⁵. It's useful for generating compiled binary outputs from your LabVIEW source in CI.

Inputs

Required:

- **MinimumSupportedLVVersion** (string): The minimum LabVIEW version to run the build with (e.g., "2018"). This must match or be lower than the LabVIEW version of the project.
- **SupportedBitness** (string): The bitness of LabVIEW to use ("32" or "64"). Typically, you may build separate 32-bit and 64-bit packed libraries.
- **RelativePath** (string): Relative path to the LabVIEW project directory or a location to use as a working directory. The build might use this to resolve project-related paths.
- **LabVIEW_Project** (string): Path to the LabVIEW project (.lvproj) that contains the build specification.
- **Build_Spec** (string): The name of the build specification within the project that produces the .lvlibp.
- **Major** (int): Major version number for the library.
- **Minor** (int): Minor version number for the library.
- **Patch** (int): Patch version number.
- **Build** (int): Build number (often used as an incrementing build counter).
- **Commit** (string): Commit identifier or build identifier (often a short git SHA) to embed or record in the build (if the build process uses it for metadata).

Optional: None (all above must be provided).

Common optional parameters like **LogLevel** and **DryRun** are accepted via the dispatcher.

Usage (Command Line)

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName build-lvlibp -ArgsJson '{
  "MinimumSupportedLVVersion": "2020",
  "SupportedBitness": "64",
  "RelativePath": ".",
  "LabVIEW_Project": "Source/MyProject.lvproj",
  "Build_Spec": "PackedLib Build",
  "Major": 1, "Minor": 0, "Patch": 0,
  "Build": 123, "Commit": "abcdef"
}'
```

This will run the Build Packed Library action. It constructs a g-cli command (using LabVIEW's CLI) to build the specified build spec (`PackedLib Build`) in `MyProject.lvproj` using LabVIEW 2020 64-bit. The version numbers (1.0.0.123) and commit id are passed into the build (often the build process or an automated versioning VI will use these to tag the output library's version and perhaps include the commit in the library metadata or file name).

The script will ensure any previous build output is removed (to avoid stale files) before building ⁴⁵, then call LabVIEW. On success, the `.lvlibp` file will be created in the configured build output directory.

Usage (GitHub Actions)

```
- name: Build Packed Project Library (64-bit)
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: build-lvlibp
    args_json: >
      {
        "MinimumSupportedLVVersion": "2020",
        "SupportedBitness": "64",
        "RelativePath": ".",
        "LabVIEW_Project": "Source/MyProject.lvproj",
        "Build_Spec": "PackedLib Build",
        "Major": 1, "Minor": 0, "Patch": 0,
        "Build": 123, "Commit": "${{ github.sha }}"
      }
```

In this example, we pass the Git commit SHA as the Commit parameter. The action will produce a 64-bit packed library. You could have a separate step for 32-bit by just changing `SupportedBitness` (and perhaps a different build spec if needed).

Outputs and Exit Codes

- **Artifacts Produced:** On success, a `.lvlibp` file (and any support files it includes) will be created in the project's build output directory. The action will log the path or name of the produced library and any version info it applied ⁴⁵.
- **Exit Code 0:** The build succeeded. LabVIEW returned a success code and the packed library is built.
- **Exit Code Non-Zero:** The build failed. A nonzero exit from LabVIEW (or an error thrown in the script) will result in the adapter throwing. For example, if LabVIEW encounters a compile error or the build spec name is incorrect, the action will fail. The script is designed to exit non-zero on failure ⁴⁶, and the adapter will propagate that. Common exit codes might be:
 - `1` for a general error (the dispatcher uses 1 for any caught exception by default ¹⁶),
 - or a specific code if the underlying CLI returns it (though typically LabVIEW CLI returns 0 or 1).
- The script also ensures to clean up any pre-existing output (deleting previous build outputs) before running ⁴⁵. If it cannot delete an old file, that might cause an error (e.g., file in use).

Logs will include the composed version string and confirmation of the build spec being run. If the build spec itself is misconfigured or fails, LabVIEW's output will be shown in the log for troubleshooting.

Dry Run Behavior

When **DryRun** is true, the adapter will **skip actually invoking LabVIEW**. It will log a message like “DryRun: would call Build_Ivlibp for project X with version 1.0.0.123” instead of running the CLI. No files will be deleted or created. Use this to verify that your parameters (project path, build spec name, version numbers) are being picked up correctly by the script.

Notes

- **LabVIEW Project & Build Spec:** Ensure the `LabVIEW_Project` path and `Build_Spec` name are correct. They must match an existing build spec inside the project file. Typos will result in a quick failure (LabVIEW won't find the spec).
- **Platform:** Must run on Windows with the specified LabVIEW version installed. Building a packed library requires LabVIEW IDE (or runtime engine with CLI support) to perform the build.
- **Multiple Builds:** You can use this action to build multiple variants (e.g., 32-bit and 64-bit). They would typically go to different output directories or have different names. The **Composite Build** action (`build`) can orchestrate both bitness sequentially (see **Build (Composite)** documentation), which might be easier than calling two separate build-ivlibp steps.
- **Cleaning:** The underlying script deletes any existing output file before building ⁴⁵. This ensures a clean output. Be mindful if your build output directory is shared among parallel processes – avoid conflicts by using separate directories or running builds sequentially.
- **Error Diagnosis:** If this action fails, check the log output for any LabVIEW dialog or error code. Sometimes build failures can be due to missing dependencies or VIs not in memory; ensure all dependencies are included in the project.

Build VI Package (Action Reference)

Action Name: `build-vi-package`

Purpose

The **Build VI Package** action automates building a VI Package (`.vip` file) using a given VIPB (VI Package Build) file. It updates the VIPB's display information (such as product version and release notes) before invoking the build, ensuring the generated package has the correct metadata ²⁶. This action uses LabVIEW's CLI to call the VI Package Manager build process (often a VI that runs the build). It is typically used after building binaries (like packed libraries) to bundle them into an installer package.

Inputs

Required:

- **SupportedBitness** (string): "32" or "64", indicating which LabVIEW bitness to use for the package build (the VIPB likely corresponds to a specific LabVIEW version/bitness).
- **MinimumSupportedLVVersion** (string): The minimum LabVIEW version needed (for loading any VIs involved in the build).
- **LabVIEWMinorRevision** (string): The LabVIEW minor revision or edition (e.g., an empty string or "f1" for LabVIEW 2020 f1). This can be used to target a specific LabVIEW patch level in the build process.
- **RelativePath** (string): A path relative to the repo, often used as the working directory context for the build (where the VIPB and relevant files are).
- **VIPBPath** (string): Path to the .vipb file (the package build specification).
- **Major** (int): Major version number for the package.
- **Minor** (int): Minor version number for the package.
- **Patch** (int): Patch version number for the package.
- **Build** (int): Build number.
- **Commit** (string): Commit or build identifier (often included in version metadata).
- **DisplayInformationJSON** (string): A JSON string (or data structure) containing display information to update in the VIPB. This typically includes fields like product name, version string, etc. The action will merge this information into the VIPB's display data.

Optional:

- **ReleaseNotesFile** (string , optional): Path to a release notes text/markdown file. If provided, the contents of this file may be read and included in the VIPB's release notes field.

(Standard LogLevel and DryRun are also supported.)

Usage (Command Line)

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName build-vi-package -ArgsJson '{
  "SupportedBitness": "64",
  "MinimumSupportedLVVersion": "2020",
  "LabVIEWMinorRevision": "f1",
  "RelativePath": ".",
  "VIPBPath": "Builds/MyProject.vipb",
  "Major": 1, "Minor": 0, "Patch": 0,
  "Build": 123, "Commit": "abcdef",
  "DisplayInformationJSON": "{ \"ProductName\": \"MyLib\", \"CompanyName\":
  \"MyOrg\" }",
  "ReleaseNotesFile": "Docs/release_notes.md"
}'
```

This would update the VIPB at `Builds/MyProject.vipb` with the given version (1.0.0.123) and commit, set the `ProductName` to "MyLib", `CompanyName` to "MyOrg" (via the `DisplayInformationJSON`), include the contents of `Docs/release_notes.md` as release notes, and then execute the package build via LabVIEW CLI ⁴⁷.

Usage (GitHub Actions)

```
- name: Build VI Package
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: build-vi-package
    args_json: >
      {
        "SupportedBitness": "64",
        "MinimumSupportedLVVersion": "2020",
        "LabVIEWMinorRevision": "f1",
        "RelativePath": ".",
        "VIPBPath": "Builds/MyProject.vipb",
        "Major": 1, "Minor": 0, "Patch": 0,
        "Build": 123, "Commit": "${{ github.sha }}",
        "DisplayInformationJSON": "{ \"ProductName\": \"MyLib\",
        \"CompanyName\": \"MyOrg\" }",
        "ReleaseNotesFile": "Docs/release_notes.md"
      }
```

Ensure that the `ReleaseNotesFile` (if used) is present. In this example, it's presumably generated or stored in the repository (maybe created by a previous step or committed).

Outputs and Exit Codes

- **Artifacts:** On success, a `.vip` file will be generated as specified by the VIPB (usually in a `dist` or `builds` directory). The VIPB file itself will also be updated (its display information fields get updated) ⁴⁸. The action prints a JSON summary of the updated display info on completion ⁴⁹.
- **Exit Code 0:** Package build succeeded. The underlying g-cli call to build the package returned 0, and the `.vip` file is created.
- **Exit Code Non-Zero:** Failure during the build. The underlying script returns the exit code from g-cli (so if the LabVIEW build VI returns an error code, that propagates) ⁵⁰. The adapter will throw on non-zero, causing a failure. Common reasons for failure: errors in the VIPB (like missing files, invalid configuration), LabVIEW not able to load something, or the `ReleaseNotesFile` not found. The script is designed to throw on error and print messages accordingly ⁵⁰.

If the step fails, inspect the log output. The JSON summary might not be printed in that case. Instead, you'll see an error message possibly from VIPM or the script.

Dry Run Behavior

With **DryRun**, the adapter will simulate the steps: it will validate inputs as usual (e.g., check if the release notes file exists, parse the JSON) but it will **not call the g-cli to build**. It may log that it would update the VIPB and call the build, but skip the actual execution. The VIPB file will not be modified in DryRun, and no .vip package will be produced. DryRun is useful to verify that your DisplayInformationJSON is well-formed and that all files are in place before doing the real build.

Notes

- **Display Information JSON:** Ensure the JSON string is properly formatted (especially when embedding in YAML – you might need to escape quotes or use the `>|` block style as shown). This JSON typically includes fields like *ProductName*, *CompanyName*, *Version*, *Description*, etc. The script will combine these with the numeric version inputs to update the VIPB. If the JSON is malformed, the script will error out.
- **Release Notes:** Providing a `ReleaseNotesFile` is optional. If given, the file's content is read and placed into the VIPB's release notes. Make sure the file is not too large or else VIPM might truncate it. If the file path is wrong, the script will throw an error (cannot find file).
- **LabVIEW CLI:** The build uses a VI (likely "Build Package.vi") via LabVIEW CLI ⁴⁷. This requires the NI VI Package Manager environment. Ensure VIPM (and the LabVIEW VI Scripting environment, if needed) is installed on the runner.
- **Post-Build Changes:** The VIPB is modified in-place with new version info. If your repository tracks the VIPB, be aware that running this action will change that file (in the runner's workspace). You might choose to commit those changes or discard them. Often, CI might not commit them, treating the version bump as ephemeral or something done in a release branch.
- **Platform:** This action requires Windows (LabVIEW). It uses file system and JSON operations in PowerShell as well ⁵¹, but those are cross-platform; the limiting factor is needing LabVIEW.

Build (Composite) – Multi-step Build Orchestrator

Action Name: `build`

Purpose

The **Build** action is a composite orchestrator that ties together multiple build steps for a LabVIEW project. Rather than performing one specific task, it automates an end-to-end build process, which typically includes: cleaning up old build outputs, building both 32-bit and 64-bit versions of a LabVIEW Packed Library, possibly closing LabVIEW in between, updating the VIPB (package build spec) with new version info, and then building the VI Package. Essentially, it's a script that uses several of the other actions internally to produce a final packaged product ²⁷. This single action can replace multiple manual steps in a workflow.

Inputs

Required:

- **RelativePath** (`string`): Path to the project's root directory (relative). Used as a base path for other operations.
- **Major** (`int`): Major version number for the release.
- **Minor** (`int`): Minor version number.
- **Patch** (`int`): Patch version number.
- **Build** (`int`): Build number (for CI/build identifier).
- **Commit** (`string`): Commit hash or identifier to include in versioning.
- **LabVIEWMinorRevision** (`string`): LabVIEW minor revision (e.g., an update or f-number) if applicable, to apply when building or packaging.
- **CompanyName** (`string`): Company or organization name (for packaging metadata).
- **AuthorName** (`string`): Author name (for packaging metadata).

Optional: None (all inputs are expected).

(This action inherently uses several underlying scripts; LogLevel and DryRun apply to the overall process.)

What It Does

Because this is a composite build process, it might help to describe what happens when you run `build`:

1. **Cleanup:** It removes old outputs/artifacts from previous builds (to ensure a clean slate) ⁵².
2. **32-bit Build:** It triggers the build of the LabVIEW Packed Library for 32-bit LabVIEW by calling the equivalent of `build-lvlibp` (with appropriate parameters for 32-bit) internally.
3. **64-bit Build:** Similarly, it builds the 64-bit Packed Library.
4. **Close LabVIEW:** After building, it may close LabVIEW to free resources or ensure no locks on files (calls the equivalent of `close-labview`).
5. **Rename Outputs:** It might rename the built files (for example, appending bitness to filenames or moving them) ⁵².
6. **Update VIPB:** It updates the VI Package Build (VIPB) file's display information with the version numbers and possibly other metadata (calls `modify-vipb-display-info` under the hood).
7. **Build VI Package:** Finally, it calls the package build (similar to `build-vi-package`) to produce the .vip file.
8. It prints out a final JSON string of display info (likely confirming the version and names) ⁵².

All these steps are done by the single `build.ps1` script, coordinating the other scripts and handling errors at each sub-step.

Usage (Command Line)

Given the complexity, using the `build` action via CLI might look like:

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName build -ArgsJson '{
  "RelativePath": ".",
  "Major": 1, "Minor": 2, "Patch": 3,
  "Build": 0, "Commit": "abcd1234",
  "LabVIEWMinorRevision": "f1",
  "CompanyName": "YourOrg",
  "AuthorName": "YourName"
}'
```

This would run the entire build orchestration for version 1.2.3.0 of the product identified by the project in the current directory. It uses commit `abcd1234` for any references, and updates Company/Author info in the package metadata. Both 32-bit and 64-bit libraries will be built and then packaged.

Usage (GitHub Actions)

```
- name: Full Build (32-bit & 64-bit -> Package)
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: build
    args_json: >
      {
        "RelativePath": ".",
        "Major": 1, "Minor": 2, "Patch": 3,
        "Build": 0,
        "Commit": "${{ github.sha }}",
        "LabVIEWMinorRevision": "f1",
        "CompanyName": "YourOrg",
        "AuthorName": "YourName"
      }
}
```

This single step will perform all build tasks. It's convenient for a release workflow – you just bump version numbers and run this, and it outputs everything needed (libraries and installer).

Outputs and Exit Codes

- **Artifacts:** The outputs of this action are the built libraries and the VI package:
- Two `.lvlibp` files (one for 32-bit, one for 64-bit) if applicable, likely placed in the `BuildOutput` or a similar directory.
- One `.vip` package file, containing the libraries and other components, ready for distribution.
- Possibly a JSON or log output of the final display information (the script prints a JSON string with version and other info on success `53`).
- **Exit Code 0:** All sub-steps succeeded and the final package is built.

- **Exit Code Non-Zero:** If any sub-step fails, the `build` action will abort and return a non-zero exit code `54`. The script is designed to exit on the first failing sub-step (it doesn't continue if, say, 32-bit build fails) `54`. The error could come from any underlying action:
- 32-bit build fail (e.g., compile error) will stop the process.
- 64-bit build fail will stop at that point.
- If those succeed but packaging fails, you'll get an error at the packaging stage.
- The logs will indicate which stage failed. Because each stage likely invokes another script, you might see that script's error message. The `build` script as a whole ensures to propagate that failure (likely through a thrown error or non-zero code).

Dry Run Behavior

When running in **DryRun** mode, the `build` orchestrator will simulate the workflow: - It will skip actual calls to LabVIEW. For each sub-step, it should log what it *would* do. For example, "DryRun: would build 32-bit Packed Library with version X", "DryRun: would build 64-bit Packed Library...", "DryRun: would update VIPB and build package...". - No files will be deleted, built, or modified. It essentially becomes a no-op that just logs the plan. - This is useful to ensure that the sequence of operations is correct and that all inputs (like CompanyName, etc.) are being read properly.

Given the complexity, DryRun helps in understanding if, for instance, both bitness builds would be triggered and if the VIPB file is found.

Notes

- **All-in-One vs Individual Steps:** This action is convenient but also somewhat inflexible. If your process differs (e.g., you don't need a 32-bit build), you might use the individual actions instead of this composite. However, if your goal is a full release build, this encapsulates the standard steps.
- **Customization:** The `build` script is built for a certain pattern of project. It assumes your LabVIEW project has both 32-bit and 64-bit build specs named consistently (perhaps it calls the `Build_lvlib.ps1` for each bitness internally). If your project doesn't need dual builds, you may ignore one of the outputs or skip steps by customizing the script (or just not using this action).
- **Order and Dependencies:** The internal order (32-bit then 64-bit) may matter if, for example, one build generates something needed for the other. Generally, they are independent, but be aware if any shared resources are touched.
- **Environment:** Must run on Windows with appropriate LabVIEW versions (likely both 32-bit and 64-bit LabVIEW installed if both builds are required). Also requires Git (for release notes generation if that's part of it) and other tooling as indirectly required by sub-actions.
- **Logging:** The final JSON output printed by the script (display info) can be captured or used for post-processing (for instance, to attach version info to a GitHub release). It's mainly informational.

Close LabVIEW (Action Reference)

Action Name: `close-labview`

Purpose

Closes a running LabVIEW instance gracefully via the LabVIEW CLI. LabVIEW, when run via automation or CLI, might stay open or hold files. This action ensures that a LabVIEW instance of a specified version/bitness is instructed to quit. It uses the CLI command **QuitLabVIEW** on the target LabVIEW installation ²⁸. This is often used in CI after performing operations like builds or adding tokens, to release file locks or simply clean up the environment.

Inputs

Required:

- **MinimumSupportedLVVersion** (string): The LabVIEW version to close (e.g., "2018"). Usually the same version you opened or used earlier in the job.
- **SupportedBitness** (string): "32" or "64" corresponding to the LabVIEW instance's bitness.

Optional: None (both must be provided).

Usage (Command Line)

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName close-labview -ArgsJson '{
  "MinimumSupportedLVVersion": "2020",
  "SupportedBitness": "64"
}'
```

This will attempt to close LabVIEW 2020 (64-bit) if it's running. The script uses g-cli to send the QuitLabVIEW command.

Usage (GitHub Actions)

```
- name: Close LabVIEW
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: close-labview
    args_json: '{"MinimumSupportedLVVersion": "2020", "SupportedBitness": "64"}'
```

Use this after steps that might leave LabVIEW open. If LabVIEW wasn't running, the CLI may still start it and then close it, or return an error indicating no instance – but typically the CLI needs to connect to do the quit command.

Outputs and Exit Codes

- **Exit Code 0:** LabVIEW closed successfully or was not running. The underlying CLI returns 0 on a successful quit, which is propagated ⁵⁵.
- **Exit Code Non-Zero:** An error occurred. Perhaps LabVIEW could not be closed (hung GUI, etc.), or the CLI encountered an issue. The script returns the g-cli's exit code ⁵⁶. The adapter will throw if that code is non-zero, causing a failure. Typically, if LabVIEW isn't open, the CLI might still return success (it just launches and closes quickly). A non-zero could mean the CLI couldn't find the LabVIEW instance (e.g., wrong version specified).

Dry Run Behavior

DryRun for close-labview will simply log that it *would* attempt to close LabVIEW with the given version/bitness, but it will not actually call the CLI. It returns immediately with success (since no action taken).

Notes

- **Use Cases:** Include this action to ensure a clean state. For instance, after building libraries or applying VIPC, LabVIEW might still be open. Closing it can free memory and allow file operations (like renaming output files) without interference.
- **Multiple Instances:** If you had multiple LabVIEW versions open (say 2019 32-bit and 64-bit), you'd need to call close-labview for each specifically. This action targets one version/bitness at a time.
- **Platform:** Windows only, as it involves LabVIEW. If run on Linux/macOS, it will throw an error (LabVIEW CLI not available).
- **No Effect:** If LabVIEW is not actually running, this action basically no-ops (or the CLI does nothing). It's safe to call even if you suspect LabVIEW might not be open, as a guard step.
- **Time-out:** The CLI usually returns quickly for Quit, but if LabVIEW takes time to close (e.g., prompt to save unsaved VIs), the CLI might hang or time out. In CI, ensure no modal dialogs in LabVIEW will block closure (run in a mode that doesn't prompt, or save changes before closing).

Generate Release Notes (Action Reference)

Action Name:

Purpose

Generates a markdown **Release Notes** file from the Git history of the repository. This action automates the collection of commit messages (or tags) to produce release notes text. It typically finds the latest tag or release in the Git history and then gathers all commits since that tag, formatting them into a markdown file ²⁹. It's useful for preparing documentation of changes as part of a release pipeline.

Inputs

Required: None.

This script operates on the repository's Git log directly. It doesn't require any parameters to function because it infers history from the Git context. (It may optionally allow an **OutputPath**, see below, but the default is used if not provided.)

Optional:

- **OutputPath** (`string`): Path to write the release notes file. If not specified, it defaults to `Tooling/deployment/release_notes.md` in the repository ⁵⁷. You can provide a custom path if you want the notes in a different location or name.

(Standard LogLevel and DryRun apply, though DryRun might simply simulate commit retrieval.)

Usage (Command Line)

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName generate-release-notes -
ArgsJson '{
  "OutputPath": "release_notes.md"
}'
```

If `OutputPath` is omitted, it will use the default. Running this will cause the script to run Git commands to gather the commit history and produce a markdown file.

Usage (GitHub Actions)

```
- name: Generate Release Notes
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: generate-release-notes
    args_json: '{"OutputPath": "release_notes.md"}'
```

Typically, you'd run this on a runner that has the git repository available (so use `actions/checkout` beforehand). The action will create or overwrite the specified markdown file. You might then choose to save this as an artifact or attach it to a release.

Outputs and Exit Codes

- **File Output:** A markdown file is created containing the release notes. The content usually includes:
 - A header with the new version or date (depending on how the script is implemented).
 - A list of commits or PR titles since the last release tag.

- Possibly grouping by categories (if commit messages follow a convention, though not guaranteed unless implemented).
- **Exit Code 0:** Notes generated successfully. This means the Git log was read and the file was written without issues.
- **Exit Code Non-Zero:** Failure in generating notes. This could happen if Git is not available or if the repository has no tags/commits (script might throw an error). The script explicitly *exits on error* and writes the file only on success ⁵⁸. If an error occurs (e.g., Git command fails), the adapter will catch it and produce a non-zero exit code.

Check the job artifact or workspace for the output file. The script's console output might also echo some of the content or at least confirm where it wrote the file.

Dry Run Behavior

In DryRun mode, the action might simulate the generation. It could e.g. log "Would generate release notes to X path" and not actually run Git or write the file. This isn't particularly useful for this action, since reading git history doesn't change state, but the DryRun ensures no file creation as a precaution.

Notes

- **Git Repository:** This action must run in a context where the `.git` folder is present (hence do a checkout). If it runs on a shallow clone or detached HEAD, it may not find tags. Consider fetching tags (`actions/checkout` has `fetch-depth: 0` option) so that the script can identify the latest tag properly.
- **Latest Tag Logic:** Typically, the script finds the latest Git tag to know where the last release was ²⁹. If your repo uses tags like `v1.2.3`, ensure they are present. If no tag is found, the script might take all commits or produce a full log.
- **Customization:** If you need a different format (e.g., include PR links or issues), you might need to adjust the script. By default, it likely just lists commit messages. You could parse conventional commit messages or similar if that was implemented.
- **Using the Notes:** After generating, you can use another step to upload the `release_notes.md` as an artifact, or use it in a GitHub Release (some workflows cat the file into the `gh release create` command, for instance).
- **Platform:** This action uses Git and file I/O – it can run on any OS with Git and PowerShell. Windows, Linux, or macOS runners are fine as long as Git is installed (on GitHub runners, it is).
- **No LabVIEW Needed:** This is one of the few actions that does not require LabVIEW. You can run it independently of the LabVIEW environment setup.

Missing in Project (Action Reference)

Action Name: `missing-in-project`

Purpose

Checks a LabVIEW project for *missing items* (such as VIs or dependencies that the project expects but cannot find). This is essentially a LabVIEW project integrity check. The action runs a LabVIEW CLI task that opens the project and uses a “Check for Missing Items” utility, then reports the results. It returns specific exit codes to indicate whether missing items were found ³⁰. This helps catch issues where code might be referencing files that aren’t present in source control or on the build machine.

There are two scripts involved: - `Invoke-MissingInProjectCLI.ps1` - the main adapter script that the dispatcher calls ⁵⁹. - `RunMissingCheckWithGCLI.ps1` - a helper that actually invokes the LabVIEW CLI with the missing items check VI ⁶⁰.

As a user of the unified interface, you only need to call `missing-in-project` action, which uses the above scripts internally.

Inputs

Required:

- **LVVersion** (`string`): LabVIEW version to use for opening the project (e.g., `"2020"`). This ensures the check is done in the right LabVIEW environment.
- **Arch** (`string`): Architecture/bitness of LabVIEW (`"32"` or `"64"`).
- **ProjectFile** (`string`): Path to the LabVIEW project (.lvproj) file to check, relative to the repository or provided working directory.

Optional: None.

Usage (Command Line)

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName missing-in-project -ArgsJson '{
  "LVVersion": "2020",
  "Arch": "64",
  "ProjectFile": "MyProject.lvproj"
}'
```

This will run the missing items check on *MyProject.lvproj* using LabVIEW 2020 64-bit. The script will open the project in LabVIEW (headless via CLI), run the check, and then exit.

Usage (GitHub Actions)

```
- name: Check for Missing Items in Project
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
```

```
action_name: missing-in-project
args_json: '{"LVVersion": "2020", "Arch": "64", "ProjectFile":
"MyProject.lvproj"}'
```

Make sure the project file is present (checked out). The action should be run on a Windows runner with the specified LabVIEW version.

Outputs and Exit Codes

The key outcomes are indicated by the exit code:

- **Exit Code 0:** No missing items were found in the project ³⁰. The project is complete; all dependencies resolved.
- **Exit Code 2:** Missing items were detected in the project ³⁰. This means some VIs or files referenced by the project could not be located. The action will still succeed in running, but it returns code 2 to signal this condition (which in CI will mark the step as “failed” unless you handle code 2 as a non-fatal outcome).
- **Exit Code 3:** An error occurred during the check ³⁰. This could be an unexpected issue (for example, the project failed to load at all, or the check VI encountered an error). This is a true error scenario.
- **Other codes:** The helper script sets `$LASTEXITCODE` accordingly. It doesn’t explicitly exit (no `Exit` call) but uses global `$LASTEXITCODE` ⁶¹. The adapter uses that to determine what to throw/return.

The action logs the list of missing items (if any) in the console. It likely prints out each missing file path or a summary. If no missing items, it might say “No missing items found.” The output is primarily textual.

In a GitHub workflow, you might treat exit code 2 differently if you want missing items to not fail the job but rather just warn. By default, a non-zero will fail the job, so you could catch it by adding `continue-on-error: true` for this step if desired, then in a subsequent step check the outcome.

Dry Run Behavior

DryRun doesn’t have much meaning here, because the action’s main effect is diagnostic (it doesn’t change anything). If DryRun is set, the adapter might skip calling LabVIEW and immediately return 0 (since it’s just a check). It would log that it would perform a missing items check on the project, but not actually do it. Use DryRun simply to test that the parameters are accepted, but note that it won’t tell you if items are missing (since it doesn’t actually open the project).

Notes

- **Headless Execution:** The missing items check runs LabVIEW in the background. Ensure no dialog requires user interaction; the scripts aim to handle everything silently.
- **Logging:** The results (missing items list) are captured and logged ⁶². You should see file paths of missing VIs in the GitHub Actions log if any. Save that output if you need to triage missing files.

- **Post-check Handling:** If missing items are found (exit 2), you might want to take certain actions: e.g., alert maintainers, or fetch those files, etc. In CI, consider whether a missing file should block the pipeline. It often should, because a build would likely fail anyway if files are missing.
- **ProjectFile Path:** Provide the path relative to your repo root or use `working_directory` input to set where to find it. For example, if your project is in a subfolder, either use `RelativePath` via the `working_directory` input of the composite action, or include the subfolder in `ProjectFile`.
- **Platform:** Windows only (requires LabVIEW). If run on non-Windows, it will error out early, likely with an “unsupported platform” message.

Modify VIPB Display Info (Action Reference)

Action Name: `modify-vipb-display-info`

Purpose

Updates the display information of a VI Package Build (VIPB) file. Display information includes fields such as product name, version, company, author, and release notes that appear in VIPM for the package. This action parses a JSON input to update these fields in the .vipb file without actually building the package. It uses a LabVIEW CLI VI (ModifyDisplayInformation) to perform the update ³¹. Use this when you want to adjust package metadata programmatically (often as part of a build process before building the package, to inject the correct version and notes).

Inputs

Required:

- **SupportedBitness** (`string`): `"32"` or `"64"` – the LabVIEW bitness to use for running the modify operation.
- **RelativePath** (`string`): Path relative to repo root, used as base directory.
- **VIPBPath** (`string`): Path to the .vipb file that should be modified.
- **MinimumSupportedLVVersion** (`string`): Minimum LabVIEW version to run the operation (and that the VIPB is associated with).
- **LabVIEWMinorRevision** (`string`): Minor revision (if any) of LabVIEW (e.g., `"f2"` for a patch).
- **Major** (`int`): Major version number to set in the VIPB display info.
- **Minor** (`int`): Minor version number.
- **Patch** (`int`): Patch version number.
- **Build** (`int`): Build number.
- **Commit** (`string`): Commit or build identifier (could be integrated into version or notes).
- **DisplayInformationJSON** (`string`): JSON string of display information fields to set (like product name, company, etc).

Optional:

- **ReleaseNotesFile** (string, optional): Path to a release notes file whose content should be placed into the VIPB's release notes field.

(LogLevel and DryRun optional as always.)

Usage (Command Line)

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName modify-vipb-display-info -
ArgsJson '{
  "SupportedBitness": "64",
  "RelativePath": ".",
  "VIPBPath": "Builds/MyProject.vipb",
  "MinimumSupportedLVVersion": "2020",
  "LabVIEWMinorRevision": "f1",
  "Major": 1, "Minor": 2, "Patch": 3,
  "Build": 0, "Commit": "abcd1234",
  "DisplayInformationJSON": "{ \"ProductName\": \"MyLib\", \"CompanyName\":
  \"MyOrg\" }",
  "ReleaseNotesFile": "Docs/release_notes.md"
}'
```

This will update *Builds/MyProject.vipb*: - Sets version to 1.2.3.0 (and perhaps forms a combined version string including commit). - Sets ProductName to "MyLib", CompanyName to "MyOrg". - Reads Docs/release_notes.md and inserts its content as the package's release notes. - Uses LabVIEW 2020 64-bit to run the operation.

Usage (GitHub Actions)

```
- name: Update Package Display Info
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: modify-vipb-display-info
    args_json: >
      {
        "SupportedBitness": "64",
        "RelativePath": ".",
        "VIPBPath": "Builds/MyProject.vipb",
        "MinimumSupportedLVVersion": "2020",
        "LabVIEWMinorRevision": "f1",
        "Major": 1, "Minor": 2, "Patch": 3,
        "Build": 0, "Commit": "${{ github.sha }}",
        "DisplayInformationJSON": "{ \"ProductName\": \"MyLib\",
        \"CompanyName\": \"MyOrg\" }",
```

```
"ReleaseNotesFile": "Docs/release_notes.md"
}
```

Usually, you'd run this right before building the actual package (VIP). After this runs, the VIPB is updated and then you can call the build (either via the `build-vi-package` action or the composite `build`).

Outputs and Exit Codes

- **Effect:** The VIPB file will be modified on disk with the new display information. If you open the VIPB in VIPM GUI after this, you should see the new product name, version, etc.
- **Exit Code 0:** The display info was successfully updated ⁶³. The script returns 0 and prints the final JSON of what it set (for confirmation) ⁶³.
- **Exit Code Non-Zero:** An error occurred. Possible issues: the VIPB file not found, the JSON not parseable, the release notes file missing, or the underlying LabVIEW VI returned an error. The action will throw and yield a non-zero exit ⁶⁴. The script returns the g-cli's exit code (which likely is 0 or 1) or throws on exceptions. Check that the file paths are correct if this fails.

Dry Run Behavior

In DryRun mode, the adapter will skip the actual modification. It will parse your JSON (to ensure it's valid) and check if files exist but will **not call LabVIEW**. It logs what it *would* set in the VIPB and then exit. The VIPB remains unchanged. Use this if you want to double-check that your DisplayInformationJSON is correctly formed and that the release notes file is accessible.

Notes

- **Partial Updates:** The JSON you provide need not include every field; the underlying VI likely only replaces the fields you specify. For instance, if you only want to update version and leave product name as-is, you could omit ProductName from DisplayInformationJSON (though in this action you likely have all info at hand).
- **Release Notes:** The content of the release notes file is typically inserted as plain text into the VIPB. Use a .txt or .md file with the text you want end-users to see.
- **Commit/Build in Version:** The action takes separate numeric version parts and a commit string. How these are combined in the VIPB depends on the implementation. Possibly the commit is appended as metadata or ignored by VIPM (since VIPM's version fields are numeric, the commit might only be used in the Product Version string or build name). The script prints a "final JSON" which likely shows exactly the fields set ⁶³, so look at that output to verify how commit was used.
- **Idempotency:** Running this action multiple times will just keep setting the same fields. It's safe to run repeatedly; it will overwrite the fields each time with the provided values (no accumulation).
- **Platform:** Requires Windows (LabVIEW). It runs a LabVIEW VI under the hood, so not available on other OS.

Prepare LabVIEW Source (Action Reference)

Action Name: `prepare-labview-source`

Purpose

Prepares the source code of a LabVIEW project for packaging or distribution by performing a special build (often called "source distribution" or similar). Specifically, this action runs a given **Build Spec** in a LabVIEW project that typically: - Removes certain tokens or configurations (e.g., disables debugging, etc.), - Produces a ZIP of the source, - Or otherwise prepares the code by executing a predefined build in LabVIEW.

It essentially automates what a developer might do to get source code ready to include in an installer or to open-source (like removing compiled code, etc.). The action uses LabVIEW CLI to run the specified build spec ³².

Inputs

Required:

- **MinimumSupportedLVVersion** (string): Minimum LabVIEW version to run (choose the LabVIEW version of the project).
- **SupportedBitness** (string): "32" or "64", depending on which LabVIEW to use.
- **RelativePath** (string): Relative path to use as working directory (likely the root of the project).
- **LabVIEW_Project** (string): Path to the LabVIEW project (.lvproj) file.
- **Build_Spec** (string): Name of the build specification in the project that prepares the source.

Optional: None (all above are mandatory).

Usage (Command Line)

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName prepare-labview-source -
ArgsJson '{
  "MinimumSupportedLVVersion": "2020",
  "SupportedBitness": "32",
  "RelativePath": ".",
  "LabVIEW_Project": "MyProject.lvproj",
  "Build_Spec": "SourceDist Build"
}'
```

This will run the build spec named "SourceDist Build" in *MyProject.lvproj* using LabVIEW 2020 32-bit. That build spec should be configured (by the project author) to perform whatever source preparation is needed (e.g., generate a ZIP of the source, remove tokens, etc.). The action will ensure the LabVIEW CLI is called with the right arguments to execute that build ³².

Usage (GitHub Actions)

```
- name: Prepare LabVIEW Source
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: prepare-labview-source
    args_json: '{"MinimumSupportedLVVersion": "2020", "SupportedBitness": "32",
"RelativePath": ".", "LabVIEW_Project": "MyProject.lvproj", "Build_Spec":
"SourceDist Build"}'
```

Run this on a Windows runner with LabVIEW. After this action, typically you would follow up with actions to package the source or to toggle into a development mode.

Outputs and Exit Codes

- **Outputs:** The specific outputs depend on the build spec's configuration. Usually:
 - A **ZIP file** of the source might be created in a known directory.
 - Certain temporary files might be produced or tokens removed from VIs.
 - The action logs any output from the LabVIEW CLI, which would include build progress or results.
- **Exit Code 0:** Build spec ran successfully to completion ⁶⁵. The source is prepared with no errors.
- **Exit Code Non-Zero:** The action encountered an error. The underlying CLI or build process likely signaled failure. The script is set to exit on error and will propagate that as a non-zero exit ⁶⁵. Reasons can include: the build spec name was not found, LabVIEW had an error building (perhaps a VI broken or missing dependency), or environment issues. Check logs for any LabVIEW error dialog output or CLI error message.

Dry Run Behavior

If DryRun is specified, the adapter will not actually invoke LabVIEW. It will log a message like “DryRun: would run build spec `<name>` on project `<path>`” and then exit with 0 (assuming inputs are valid). This does not create any outputs and is only for verification of parameters.

Notes

- **Preconditions:** The LabVIEW project should have a defined build spec intended for preparing source. Ensure that exists and is named exactly as given. This action doesn't create any build spec; it just calls the one you name.
- **Postconditions:** After running, if the purpose was to create a source distribution (e.g., a zip of source code with certain settings), you can use the output in subsequent steps (like attach the ZIP as an artifact or include it in a package).
- **Tokens:** The description mentions removing tokens ³². Likely the build spec or script ensures any *dev tokens* (like enabling debug, etc.) are removed from the VIs so that the source is clean. If your process requires adding tokens first (to allow building with no dialogs), ensure those steps happen (e.g., maybe **Set Development Mode** adds tokens, then this prepare uses them).

- **Pair with Restore:** There is a corresponding **restore-setup-lv-source** action that likely reverts what this prepare does. Typically, you *prepare* source (removing tokens, unzipping, etc.) when entering a dev environment, and *restore* it after you're done (to go back to packaged form). Keep this in mind for workflows; the Set Development Mode and Revert Development Mode actions also orchestrate these.
- **Platform:** Windows only (needs LabVIEW to run the build spec).
- **Troubleshooting:** If this fails, try manually running the build spec in the LabVIEW IDE to see if it requires any interactive steps or if any VI is broken. The CLI might not give detailed errors beyond a code.

Rename File (Action Reference)

Action Name: `rename-file`

Purpose

Renames a file on the filesystem. This is a simple utility action to change a file's name or move it to a new path (since a rename can also be used to move a file). It's included to handle file operations within workflows, often needed after builds (for example, renaming output files for consistency). The action is essentially a wrapper around PowerShell's `Rename-Item` cmdlet, with added error handling to avoid overwriting files unexpectedly ³³.

Inputs

Required:

- **CurrentFilename** (`string`): The current path/name of the file to be renamed. This can be relative to the working directory or an absolute path.
- **NewFilename** (`string`): The new path/name for the file. This can include a different directory if moving the file.

Optional: None.

Usage (Command Line)

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName rename-file -ArgsJson '{
  "CurrentFilename": "builds/old_name.txt",
  "NewFilename": "builds/new_name.txt"
}'
```

This will attempt to rename `old_name.txt` to `new_name.txt` in the *builds* directory.

Usage (GitHub Actions)

```
- name: Rename output file
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: rename-file
    args_json: '{"CurrentFilename": "builds/MyLib32.lvlibp", "NewFilename":
"builds/MyLib_32bit.lvlibp"}'
```

For example, after building a 32-bit library and a 64-bit library with the same name, you might rename one of them to include the bitness in the name.

Outputs and Exit Codes

- **Effect:** The file at `CurrentFilename` is renamed/moved to `NewFilename`. If `NewFilename` exists already, the script likely throws an error to avoid overwriting (unless it's explicitly coded to overwrite; but given typical safety, it might prevent overwriting).
- **Exit Code 0:** File successfully renamed ⁶⁶.
- **Exit Code Non-Zero:** Failure occurred. Common reasons:
 - The `CurrentFilename` does not exist (file not found).
 - The target `NewFilename` is on a different volume or in use (standard Rename-Item errors).
 - A file with the `NewFilename` already exists and the script decided to treat that as an error. The script is said to have error handling ³³, so likely it catches exceptions from Rename-Item and then returns a non-zero code instead of silently overwriting.

The adapter will catch errors and log them, and the dispatcher will output them as well.

No structured output is produced (just the file operation). If successful, the logs may note the rename.

Dry Run Behavior

DryRun for `rename-file` will log what it would do (e.g., “Would rename X to Y”) but not actually perform the operation, then exit with 0. This ensures you can test the step without affecting the filesystem.

Notes

- **Case Sensitivity:** On Windows, file renames are case-insensitive (renaming just the case might not change the file). On Linux, the action could rename case. But since this is mostly for Windows usage (given the context), just be aware that renaming “file.txt” to “File.txt” might not do anything on Windows.
- **Overwrite:** If you do intend to overwrite, you should manually handle deletion of the target first. This action by design doesn't mention an option to force overwrite, to avoid mistakes. So ensure the target name is free.
- **Path issues:** If either path is invalid or points to a directory rather than file, the action will error. It's not meant for moving directories (just files).

- **Cross-Platform:** This action can actually run on any OS with PowerShell Core since it's just file operations. If using on Linux/macOS, ensure paths are correct (and consider using forward slashes or escaping). But typical use is on Windows.
- **Idempotency:** If you run this action again on the same inputs after it succeeded once, it will likely error (since the original file no longer exists, and the new name already exists). So it's not idempotent unless you clean up or check existence in advance. The scripts might not check if the new name already exists except by letting Rename-Item throw.

Restore LabVIEW Source (Action Reference)

Action Name: `restore-setup-lv-source`

Purpose

Restores a LabVIEW project's source code from a packaged state back to a development state. In essence, it reverses what **Prepare LabVIEW Source** does. If prepare zipped or modified the source, restore will unzip it and reapply tokens as needed to return the source to a usable state for development. This action uses the same parameters as prepare and likely calls a complementary build spec or a series of operations to undo the packaging ⁶⁷. It's typically used to switch a repository *out* of "development mode" after you have finished building.

Inputs

Required: (Same as **prepare-labview-source**)

- **MinimumSupportedLVVersion** (`string`): LabVIEW version to use for the restore operations.
- **SupportedBitness** (`string`): "32" or "64", for the LabVIEW instance.
- **RelativePath** (`string`): Relative path to the project.
- **LabVIEW_Project** (`string`): Path to the LabVIEW project file.
- **Build_Spec** (`string`): Name of the build spec (or in this case possibly the "restore" spec, often the same name or a corresponding spec) to run for restoration.

Optional: None.

Usage (Command Line)

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName restore-setup-lv-source -
ArgsJson '{
  "MinimumSupportedLVVersion": "2020",
  "SupportedBitness": "32",
  "RelativePath": ".",
  "LabVIEW_Project": "MyProject.lvproj",
```

```
"Build_Spec": "SourceDist Build"
}'
```

This assumes that the same build spec used to prepare (for example, creating a zip) can be used or has a counterpart to restore. In some setups, the restore might simply involve unzipping an archive that prepare created and removing tokens. It's possible the script directly handles the unzip and token removal without an explicit LabVIEW build spec, given that the description suggests accepting the same parameters but then performing the reverse operation ³⁴.

Usage (GitHub Actions)

```
- name: Restore LabVIEW Source
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: restore-setup-lv-source
    args_json: '{"MinimumSupportedLVVersion": "2020", "SupportedBitness": "32",
"RelativePath": ".", "LabVIEW_Project": "MyProject.lvproj", "Build_Spec":
"SourceDist Build"}'
```

Typically, this would be run after you've finished the build and possibly distributed the source. For example, if you had set development mode and then built, you might restore to put everything back in the state it was (repackage sources into library form etc.).

Outputs and Exit Codes

- **Effect:** The project's source is restored. This could mean:
 - If `prepare` had unzipped sources into directories, `restore` might remove those and put back the packaged library files.
 - If `prepare` removed tokens from VIs, `restore` might re-add them (like re-enabling certain settings).
 - Essentially the project is returned to the state before `prepare` was run.
- **Exit Code 0:** Restoration succeeded with no errors ⁶⁸.
- **Exit Code Non-Zero:** Something went wrong. Perhaps the expected archive or files to restore from were not found. For example, if there's no zip to unzip, or some step in re-tokenizing failed. The script will print an error and exit non-zero ⁶⁸. This should be rare if used correctly (and likely the script checks for existence of what it needs).

Dry Run Behavior

DryRun will log what it intends to do (e.g., "DryRun: would restore source for project X by unzipping Y and removing tokens") and then not perform any changes. Useful for verifying that it identifies the right files to act on.

Notes

- **Symmetry with Prepare:** Typically, you use `prepare-labview-source` before building (to get raw source ready), and after you're done (or perhaps after packaging) you use `restore-setup-lv-source` to clean up. If you are using **Set Development Mode** and **Revert Development Mode** actions, those actually call these internally for both bitness in sequence ³⁵ ⁶⁹ . So you might not need to call prepare/restore directly if you use the higher-level dev mode actions.
 - **Parameters:** It uses the same inputs as prepare to know what to restore. Likely it assumes the `Build_Spec` parameter points to some known procedure; in some setups, it might not actually use the spec for restore but uses it as an identifier.
 - **Safe to Run:** If you run restore when nothing is in development mode (e.g., if sources are already packaged), the script should handle it gracefully (maybe nothing to do, or error out if it expected something). It's best used only after a corresponding prepare.
 - **Environment:** Needs LabVIEW if it calls any VIs or uses CLI to close projects, etc., but much of restore might just be file operations (like unzipping) which PowerShell can handle. However, since it's still under g-cli category, assume LabVIEW and tools are needed.
 - **File Modifications:** This action will modify files in your repo (unzipping will write files, removing tokens will alter config or VIs). If you are doing this on a Git working copy, be aware that it might leave the working copy with changes (if you care to keep it clean). Many CI flows just revert or discard those changes at end.
-

Revert Development Mode (Action Reference)

Action Name: `revert-development-mode`

Purpose

Reverts a repository from "development mode" back to its packaged or release state. This is a high-level orchestrator that calls **Restore LabVIEW Source** for both 32-bit and 64-bit, and then closes LabVIEW. Essentially, if **Set Development Mode** opened up the project for development (by unpacking source, adding tokens), this action will do the inverse: remove development tokens, repackage the source into libraries, and close any open LabVIEW instances ³⁵ . It ensures that after it runs, the repository is back in a state ready for distribution or normal usage (not in a dev-unpacked state).

Inputs

Required:

- **RelativePath** (`string`): Path to the repository or project root (relative). This is used to locate project files and to operate in the correct directory.

Optional: None (the action knows what fixed procedures to run internally; no other input needed).

Usage (Command Line)

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName revert-development-mode -
ArgsJson '{
  "RelativePath": "."
}'
```

This will run the revert process for the project located at the current directory. Internally, it will probably: - Navigate to the RelativePath directory. - For 32-bit: call RestoreSetupLVSource (with appropriate arguments for that bitness). - For 64-bit: call RestoreSetupLVSource (for 64-bit). - Then call Close LabVIEW to shut down any LabVIEW instances.

The script likely uses a helper like `Execute-Script` to run those sub-actions safely ⁶⁹.

Usage (GitHub Actions)

```
- name: Revert Development Mode
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: revert-development-mode
    args_json: '{"RelativePath": "."}'
```

Typically used at the end of a build job. For example, if you had previously done `set-development-mode` at the start of the job to get source in a modifiable state, after building and packaging, you do `revert-development-mode` to tidy everything up.

Outputs and Exit Codes

- **Effect:** The repository is returned to packaged state:
- Any source files that were unzipped or extracted might be removed.
- Packed libraries (like .lvlibp) that were removed in dev mode might be put back or remain.
- Development tokens added to LabVIEW are removed (closing LabVIEW also helps ensure no dev tokens linger in memory).
- Essentially, you end up with the same file set as before dev mode was set.
- **Exit Code 0:** Successfully reverted to normal mode ⁷⁰. All sub-steps succeeded (both bitness restores and LabVIEW closed without issue).
- **Exit Code Non-Zero:** A failure happened in one of the steps ⁷⁰. The script will stop and return an error if, say, the restore for 32-bit failed or the close-labview failed. It is orchestrated such that any failure along the way bubbles up. You'll see in logs at what step it failed. For instance, if 64-bit restore fails (maybe because 64-bit wasn't in dev mode or files missing), the process stops there.

Dry Run Behavior

DryRun mode will simulate the process: log that it would call restore for 32-bit, restore for 64-bit, close LabVIEW, but not actually do any of it. Good for verifying the sequence without touching the system.

Notes

- **Prerequisite:** Ideally, run this only after `set-development-mode` was run and you've completed your development tasks (like building). If dev mode was never set, running revert might not do much, or could error if it expects some files to exist.
 - **What it calls:** Under the hood, it calls `RestoreSetupLVSource.ps1` for both bitness using the same RelativePath and build spec (the script probably knows which build spec or uses default naming) ³⁵, then `Close_LabVIEW.ps1` ⁶⁹. The mention of *Execute-Script* suggests it might use a helper to run those and capture errors without stopping immediately on the first (ensuring both attempts run, but that's speculation from the note).
 - **After Effects:** Once reverted, your repo is likely back to having built artifacts instead of raw source (if that's how it started). If your pipeline is done, you might not care, but if you plan to run tests after packaging, you'd likely run them before reverting (since reverting might remove the unpacked test VIs).
 - **Cleaning:** This is a cleanup step. It's a good practice to always call it in a `finally` /post-job so that even if builds fail, LabVIEW is closed and the environment reset. Otherwise, a runner machine might have LabVIEW left open or temp files lying around.
-

Run Unit Tests (Action Reference)

Action Name: `run-unit-tests`

Purpose

Runs automated unit tests for a LabVIEW project using the LabVIEW Unit Test Framework via CLI (or a similar mechanism). It opens the project, runs all or a set of tests, and then produces a report. The action parses the UnitTestReport.xml that LabVIEW generates to summarize results, printing a formatted table of test outcomes, and sets an appropriate exit code depending on whether tests passed, failed, or an error occurred ³⁶. This allows CI to mark the build as failed if any tests fail (with a distinct code) or if the test run itself errors out.

Inputs

Required:

- **MinimumSupportedLVVersion** (`string`): LabVIEW version to use for running tests (e.g., `"2020"`).

- **SupportedBitness** (string): "32" or "64" corresponding to the LabVIEW instance for the tests.

(The action presumably finds the project and test information from context. Possibly it looks for a .lvproj in the working directory or has conventions for the test harness name, since no explicit project path is given here in inputs. The script might assume that RelativePath is the working directory or uses the one provided via -WorkingDirectory if needed.)

Optional: None (besides common options).

Usage (Command Line)

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName run-unit-tests -ArgsJson '{
  "MinimumSupportedLVVersion": "2020",
  "SupportedBitness": "64"
}'
```

This will execute the unit tests for the LabVIEW project in the current directory (assuming the .lvproj is found there or known). The action likely expects the project's unit tests to be configured (the Unit Test Framework tests are usually defined in a .lvttest or within the project file). It runs them via CLI, which generates an XML report.

Usage (GitHub Actions)

```
- name: Run LabVIEW Unit Tests
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: run-unit-tests
    args_json: '{"MinimumSupportedLVVersion": "2020", "SupportedBitness": "64"}'
```

Make sure the working directory for this step is set to the folder containing your LabVIEW project, or that the action can find the project file. You may use the `working_directory` input of the composite action if your project is in a subfolder.

Outputs and Exit Codes

- **Test Report:** The Unit Test Framework typically produces an `UnitTestReport.xml`. The action reads this, and prints a summary table to the console of test names and their status (Passed/Failed) ⁷¹. It may also note how many passed vs failed.
- **Exit Code 0:** All tests passed successfully ³⁶ ⁷². This means no unit test reported failure.
- **Exit Code 2:** One or more tests failed, but the test run executed completely ³⁶ ⁷². The action uses code 2 to indicate a test failure situation (distinct from an outright error). This allows differentiation: CI can treat this as “tests failed” but the job will still be marked failed unless caught. The log table will show which tests failed.

- **Exit Code 3:** An error occurred during the test run execution itself ³⁶ ⁷² . This could be due to the project not loading, the Unit Test framework not found, or a crash during tests. It indicates that the tests did not fully run to completion.
- The script ensures to clean up any test output directories it created (it likely creates some temporary directories for running tests) ⁴ .

GitHub will mark the step as failed for exit code 2 or 3. You might specifically want to interpret 2 differently; typically, 2 is still a failure (just semantic to know “tests failed” vs “script error”).

Dry Run Behavior

If DryRun is enabled, the action will not actually run tests. It might immediately output a message that it would run tests on the given LabVIEW version/bitness and then exit with 0 (since no tests run means no failures by definition in simulation). This is mainly to verify that the action is configured right, not to test your tests.

Notes

- **Test Identification:** By default, the action likely runs *all* unit tests defined in the project. If you have multiple .lvproj or specific test configurations, you might need to ensure the correct project is in context. Possibly it assumes a naming convention like a test harness VI or a particular project structure. Check how your tests are set up.
- **Dependencies:** The Unit Test Framework must be available in the LabVIEW installation on the runner. If not, running tests will not work. This is an NI add-on, ensure it's installed.
- **Output Artifacts:** You may want to archive the UnitTestReport.xml as a CI artifact or use a test reporting action to interpret it. The file is usually in the project directory or a LabVIEW temp directory. The action deletes the intermediate outputs, but likely leaves the XML for you to see ⁴ .
- **Interpretation:** The action nicely maps test outcomes to exit codes as described, so you can rely on the CI failing if any tests fail. If you want the job not to fail on test failures (perhaps to gather results), you'd have to handle exit code 2 specially (maybe with `continue-on-error` and then parse results).
- **Platform:** Windows only (requires LabVIEW). The test framework isn't available on other OS in most cases.
- **Large Test Suites:** Running many tests can take time. You might see LabVIEW open in the background. The action prints a table; if there are many tests, maybe it truncates or just summarizes. Check the full XML for details on each test if needed.

Set Development Mode (Action Reference)

Action Name: `set-development-mode`

Purpose

Configures a LabVIEW project's repository for **development mode**. This usually means transforming the repo from a state where source might be packaged (e.g., in .lvlibp libraries) to a state where everything is unpacked and editable. The action performs multiple steps: it removes any existing build artifacts (packed libraries, etc.), adds necessary tokens to LabVIEW (to enable certain dev features), prepares the source by unzipping or extracting it, and then closes LabVIEW ³⁷. In short, it sets up the environment so a developer (or CI process) can operate on the raw VIs and code.

It leverages other scripts: e.g., calls `AddTokenToLabVIEW`, `Prepare_LabVIEW_source`, etc., possibly for both 32-bit and 64-bit. It loops through bitness to ensure both versions of the project are ready for dev ⁷³.

Inputs

Required:

- **RelativePath** (string): Path to the project/repo root (relative). This is used to locate the project and as the working directory for operations.

Optional: None.

Usage (Command Line)

```
pwsh -File actions/Invoke-OSAction.ps1 -ActionName set-development-mode -
ArgsJson '{
  "RelativePath": "."
}'
```

Running this will: - Delete existing compiled libraries or artifacts (so the source can be used in place). - Add tokens to LabVIEW (like setting it to skip mass compile dialogs or allow multiple app instances, depending on need). - Prepare the source by running the special build spec to extract source (for both 32-bit and 64-bit if applicable). - Likely open and close LabVIEW in the process.

After completion, the repository is in “development mode,” meaning you have all VIs accessible for editing or for running unit tests.

Usage (GitHub Actions)

```
- name: Set Development Mode
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: set-development-mode
    args_json: '{"RelativePath": "."}'
```

This is typically one of the first steps in a CI job, before building or testing. It ensures the environment is configured for subsequent actions like building or testing.

Outputs and Exit Codes

- **Effect:** The repository is now in dev mode:
- All previous build artifacts (like .lvlibp files or perhaps .vip packages) are removed from the source directories.
- LabVIEW's INI has dev tokens (like "SecretPanel=True" or similar) added, allowing operations that might be required for building from source.
- The source code that was inside packed libraries is extracted out (you might see new files or directories with the source VIs).
- LabVIEW is closed at the end, leaving the environment ready.
- **Exit Code 0:** Successfully entered development mode ⁷⁴. This means all sub-steps succeeded (token addition, extraction, etc.).
- **Exit Code Non-Zero:** A failure occurred in one of the steps ⁷⁵. Possibly:
 - LabVIEW not installed or CLI failed to add tokens,
 - The prepare step failed (maybe the build spec to prepare source had an issue),
 - Something about file operations (like deletion) failed due to permission. The script will stop at the failing sub-step and return an error. Logs will indicate which part failed.

Dry Run Behavior

DryRun will simulate the steps: - It will not actually delete files or add tokens or extract source. - It will log what it *would* do: e.g., "Would remove packed libraries X, Y", "Would add tokens A, B to LabVIEW", "Would run prepare source for 32-bit and 64-bit", etc. - Then exit with 0. This is useful to see what changes it intends to make, without touching anything.

Notes

- **Scope:** This action touches both 32-bit and 64-bit contexts of the project (if applicable). It likely calls AddTokenToLabVIEW for maybe both bitness or just once (the token probably applies to both). It then calls Prepare_LabVIEW_source for 32-bit and for 64-bit (the doc mentions it loops through bitness ⁷³).
- **Cleanup:** It removes built libraries. If those libraries are tracked in your repo, note that your working copy will have deletions. In CI, that's fine; on a local dev machine, use with caution as it might delete built artifacts you haven't backed up (though presumably those can be rebuilt).
- **Idempotency:** Running set-development-mode when already in dev mode should ideally do nothing new (maybe just re-add tokens which are already there, which is fine, and ensure no libraries exist which they don't). It should be largely idempotent, aside from possibly trying to delete already deleted files (which shouldn't error if properly handled).
- **When to run:** Use it at the start of a job that needs to run tests or analysis on source. After you finish, use **revert-development-mode** to put things back.
- **LabVIEW State:** Adding tokens modifies LabVIEW INI, which persists beyond the run. If on a self-hosted runner, those tokens remain until removed (revert might remove them by closing LabVIEW or explicitly?). Keep in mind persistent state if not ephemeral runner.

- **Examples of tokens:** Not explicitly listed here, but tokens could be things like "Skip unit test result dialogs" or enabling certain private VI server permissions required for operations.
 - **Close LabVIEW:** It closes LabVIEW at the end to ensure a clean state (and possibly to flush any changes like tokens into the INI file). This is good to avoid conflicts with later steps that also need to run LabVIEW CLI.
-

Developer Guide

For contributors who want to add new actions (adapters) or modify existing ones in the Open-Source Actions project.

This guide explains the structure of the unified dispatcher and how to extend it with new adapters and scripts. By following the conventions below, your new action will integrate smoothly with the dispatcher, composite action, and documentation.

Architecture Recap

Before adding a new adapter, it's important to understand the components:

- **Dispatcher Script** (`Invoke-OSAAction.ps1`): The entry point that parses inputs and dispatches to an adapter function based on `-ActionName`. It contains a registry (hashtable) mapping action names to function names ⁹. It handles common concerns like JSON parsing, setting working directory, and global error catching ⁷⁶ ¹⁰.
- **PowerShell Module** (`OpenSourceActions.psm1`): This module houses all adapter functions (one per action) and any shared logic. Adapters are named `Invoke<Action>` (e.g., `InvokeAddTokenToLabVIEW`) and are exported via the module manifest ⁷⁷.
- **Underlying Scripts:** Each action has an underlying script (or set of scripts) that actually implement the functionality (usually existing in the `actions/<action-name>/` folder). The adapter will call this script.
- **Composite Action** (`action.yml` in `abstract-action/`): Wraps the dispatcher for GitHub workflows, exposing inputs that mirror `Invoke-OSAAction.ps1` parameters ⁷⁸ ⁷⁹.

When adding a new action, you will create/modify elements in several of these components.

Naming Conventions

Action Name (string): Should be all-lowercase with words separated by hyphens, e.g., `"my-new-action"`. Use a concise, descriptive name that fits the pattern of existing actions. Typically, it matches the folder name where the script resides.

Script Filename: The underlying script (PowerShell `.ps1`) implementing the action's logic should go into `actions/<action-name>/` folder. If possible, name the script clearly (e.g., `MyNewAction.ps1`). If the script is an existing one you are onboarding, keep its name but ensure the folder name and ActionName reflect a friendly kebab-case name.

Adapter Function Name: In the module, name it `Invoke<PascalCase>` version of the action. For example, `"my-new-action"` -> function `InvokeMyNewAction`. Use PascalCase for multi-word names, preserving meaningful capitalization (e.g., `VIPC` stays uppercase as in `InvokeApplyVIPC`). The function name should ideally correlate obviously to the script name.

Folder and File Structure: Create a new folder under `actions/` if not already present:

```
actions/  
  my-new-action/  
    MyNewAction.ps1
```

(You might also include any helper scripts here if needed.)

Update the repository's **registry** in `Invoke-OSAAction.ps1` by adding an entry mapping `'my-new-action'` to `'InvokeMyNewAction'` ⁸⁰.

Adapter Function Template

Add your function to `OpenSourceActions.psm1`. Follow this pattern:

```
function InvokeMyNewAction {  
  [CmdletBinding()]  
  param(  
    # Required parameters  
    [Parameter(Mandatory)] [string] $Param1,  
    [Parameter(Mandatory)] [int] $Param2,  
    # ... other required params  
    # Optional parameters  
    [Parameter()] [string] $OptionalParam = "default",  
    [Parameter()] [string] $LogLevel = 'INFO',  
    [switch] $DryRun  
  )  
  Write-Information "Invoking MyNewAction with Param1=$Param1 ..." -  
InformationAction Continue  
  $scriptPath = Join-Path $PSScriptRoot 'my-new-action/MyNewAction.ps1'  
  # Build argument hashtable for underlying script  
  $args = @{  
    Param1 = $Param1  
    Param2 = $Param2  
    OptionalParam = $OptionalParam  
    # ... include all script params except LogLevel/DryRun  
  }  
  if ($DryRun) {  
    Write-Information "DryRun: would call $scriptPath with args $($args |  
ConvertTo-Json -Compress)"
```

```

        return 0
    }
    & $scriptPath @args
    if ($LASTEXITCODE -ne 0) {
        throw "MyNewAction failed with exit code $LASTEXITCODE"
    }
    return $LASTEXITCODE
}

```

Parameter Block: - Use `[CmdletBinding()]` to get advanced function features. - Mark required inputs with `[Parameter(Mandatory)]`. - Provide types (use `[string]`, `[int]`, `[bool]` / `[switch]`, etc., matching what the script expects). - Include common optional parameters: - `$LogLevel` (string, default 'INFO') - even if your adapter doesn't explicitly use it, including it ensures consistency (the dispatcher will set `$VerbosePreference` based on it ¹⁸). - `$DryRun` (switch) - allow simulation.

Calling Underlying Script: - Use `Join-Path $PSScriptRoot` to locate the script file relative to the module, as shown above. This ensures it finds the script regardless of current directory. - Prepare a hashtable `$args` with keys exactly matching the underlying script's param names. Fill it from adapter params. - Do not include `LogLevel` or `DryRun` in `$args` for the script unless the script explicitly has those parameters (most underlying scripts do not, they are older scripts). Instead, the adapter handles `DryRun` by not calling the script.

DryRun Handling: - If `$DryRun` is set, log what *would* happen (use `Write-Information` for consistency). Typically, log the script path and arguments. Then `return 0` to exit successfully without doing anything. - Make sure to do this **before** calling the script (see template).

Executing Script: - Use the call operator `&` with the `$args` splatted (`& $scriptPath @args`). This executes the script with those parameters. - After execution, check `$LASTEXITCODE`. If it's non-zero, throw an exception with a helpful message. Throwing is important: it triggers the dispatcher's catch, leading to a controlled failure and proper exit code handling ⁸¹. - If `$LASTEXITCODE` is zero, simply return it (or just return 0).

Logging in Adapter: - Use `Write-Information` for normal logs (respecting `InformationPreference`, which the dispatcher ties to `LogLevel`). For critical errors inside adapter (other than script exit codes), you might `Write-Error` or just throw. - Avoid using `Write-Host` or direct output - stick to `Write-Information/Verbose` for logs so they can be filtered by `LogLevel`.

Mandatory Parameter Validation: - If an underlying script expects certain combos, handle that either in script or adapter. E.g., if some params are interdependent, you can add logic in adapter to throw if invalid combination.

After writing the function: - Add it to the `FunctionsToExport` list in `OpenSourceActions.psd1` (module manifest) so that it's available when module is imported ⁸². - Add the `ActionName` mapping in `Invoke-OSAction.psm1` registry (as mentioned). - Include your function in alphabetical or logical order among others in the `.psm1` for neatness.

Registry Update (Invoke-OSAction.ps1)

Open `Invoke-OSAction.ps1` and find the `$registry = @{ ... }` hashtable ⁹. Add an entry for your action:

```
$registry = @{  
    'add-token-to-labview' = 'InvokeAddTokenToLabVIEW'  
    'apply-vipc'           = 'InvokeApplyVIPC'  
    # ... existing mappings  
    'my-new-action'       = 'InvokeMyNewAction'  
}
```

Make sure the key is lowercase (the dispatcher does `.ToLower()` on `ActionName` before lookup ⁸³) and the value is exactly your function name.

No other parts of `Invoke-OSAction` usually need changing, unless your action needs special global handling (rare).

Logging and Verbosity

The dispatcher sets `$VerbosePreference` and `$InformationPreference` based on the `-LogLevel` input. In your adapter or scripts, use: - `Write-Verbose "details"` for debug-level logs. - `Write-Information "info"` for general info. - `Write-Warning "warn msg"` if something is concerning but not fatal. - `Write-Error "error msg"` for errors (though in adapters usually you throw instead of `Write-Error`, to stop execution).

This way, if user sets `LogLevel` to `DEBUG`, they'll see verbose logs. At `INFO`, they'll see information messages.

DryRun Considerations

Ensure that any destructive or external-call action in your adapter is guarded by `DryRun`. In many cases, simply not calling the underlying script suffices, as the script does the real work. If your adapter itself had multiple steps, wrap each in `if (!$DryRun) { ... }` as needed, but often just the main call is enough.

Also, your underlying script might support a `-WhatIf` or similar. We generally don't rely on that; we handle `DryRun` at the adapter level.

Test `DryRun` by running the dispatcher with your action and `-DryRun` to see that it logs the intended message and does nothing else.

Testing Your Adapter

Pester Tests: Add or update tests in `tests/pester/` for your adapter. Create a `.Tests.ps1` file if not exists. Write tests to cover: - Adapter function accepts valid input and calls underlying script (you might mock the script call). - Mandatory parameter enforcement (calling without a required param should throw). - DryRun returns success without executing script (you can set up a dummy script that would create a file, and ensure DryRun didn't create it, for example). - If possible, test that a non-zero \$LASTEXITCODE from script leads to the adapter throwing.

Because some adapters call LabVIEW, full integration tests might be tricky. Use mocking where appropriate or limit to logic tests.

Smoke Testing: On a real system (or a GitHub runner), perform a manual run of the new action: - Use the composite action in a workflow or run `pwsh Invoke-OSAAction.ps1 -ActionName my-new-action ...` with sample args. - Ensure it does what you expect (check files, outputs, exit code). - Test error cases: give bad input to see if it fails gracefully with a clear message.

Module Version and Documentation

If your new action is a significant addition, consider bumping the module version (MINOR, typically) in the `.psd1` and adding an entry to release notes. See [docs/versioning.md](#) for version policy.

Document the new action: - Create a markdown file in `docs/actions/` for it, following the format of others, so users know how to use it. - Update any central lists (e.g., the README's list of actions) to include it.

By maintaining documentation and tests alongside the code, we keep quality high.

Logging and Error Handling Guidelines

- **No Plain Write-Host:** Use structured logging (Information/Verbose/Warning/Error).
- **Throw on Failure:** Adapters should `throw` on any condition where the action didn't accomplish its task. This triggers the unified error handling (the dispatcher will catch it and convert to a standardized error message with exit code 1) ¹⁶.
- **Exit Codes:** Under normal operation, do not call `Exit` in adapters. Just return \$LASTEXITCODE or throw. The dispatcher calls `exit $exitCode` at the very end for you ¹⁶. Underlying scripts might call `exit` or set \$LASTEXITCODE; our pattern is to avoid explicit `exit` in adapters.
- **Cleaning Up:** If your adapter creates any temp files or changes state, try to revert it or document that the action does this. The framework doesn't have a formal rollback, but for example, if you set an environment variable or registry, consider resetting it in a `finally` if an error happens. Most actions won't need this.

Following these Steps

By adhering to these conventions, your new action will be: - Discoverable via `Invoke-OSAction - ListActions` (since it's in the registry). - Usable in GitHub Actions via the `action_name` input. - Consistent in error handling and logging behavior with other actions. - Easier to maintain and debug.

Happy coding, and thank you for contributing to Open-Source Actions! Make sure to run the full test suite (`Invoke-Pester`) and update documentation before submitting your changes.

Design Rationale

Why a Unified Dispatcher?

Originally, this repository contained numerous independent PowerShell scripts for CI tasks (adding LabVIEW tokens, building packages, running tests, etc.), each with its own interface and assumptions. This led to duplicated logic, inconsistent parameter names, and difficulty in orchestrating complex flows. The unified dispatcher architecture was introduced to provide a **decoupled abstraction** for these tasks ¹. Instead of calling scripts directly, callers use a single entrypoint (`Invoke-OSAction.ps1`) with an action name and JSON arguments. This indirection allows: - Consistent invocation and error handling for all actions. - Easier updates: the underlying script can change or move, but as long as the adapter is updated, external calls remain the same. - Discoverability and self-documentation of available actions through the dispatcher.

Separation of Concerns – Dispatcher, Adapters, Leaf Scripts:

The design cleanly separates the **dispatcher**, **adapter layer**, and **leaf scripts**: - The *dispatcher* (`Invoke-OSAction.ps1`) is a thin coordinator. It loads the module, parses JSON input, selects the right adapter, and catches any errors to set the final exit code ¹⁵ ¹⁶. - The *adapters* (functions in `OpenSourceActions.psm1`) act as intermediaries. Each adapter has a strong parameter schema (enforcing required vs optional, proper types) and is responsible for invoking the corresponding leaf script and handling its result ¹³ ⁸⁴. Adapters translate generic JSON input into concrete script parameters. - The *leaf scripts* (in `actions/<name>/*.ps1`) contain the actual implementation for the CI task (e.g., calling LabVIEW CLI commands, file operations). They were often written earlier and expect certain PowerShell parameters and `$LASTEXITCODE` usage. The adapter wraps these without modifying their internal logic, preserving proven behavior while adding a uniform interface on top ⁸⁵.

This separation means each part can be worked on independently: - We can improve logging or error handling in adapters and it benefits all actions. - We can update a leaf script (say to support a new LabVIEW version) without affecting how callers invoke it (since the adapter's interface stays the same). - The dispatcher remains simple, mostly configuration via the registry.

Discovery and Self-Documentation:

Having many actions raises the question: how do users know what actions exist and what parameters to send? The design anticipated adding discovery commands. The dispatcher could support: - `-ListActions`: to list all action names available ²². This enumerates the registry and perhaps prints each action with a brief description. - `-Describe <ActionName>`: to show details of one action, e.g., required/optional parameters and maybe default values.

This wasn't a necessity for initial functionality, but it's a natural extension. It can be implemented by reflecting on the adapter function's param block or by maintaining metadata in a dictionary. The rationale is to improve usability – users can run a help command rather than digging into documentation or code to figure out usage.

Even without those commands implemented yet, the unified structure makes documentation easier: we document one interface (Invoke-OSAction with action name and JSON) and list of actions, instead of documenting 16 separate scripts with different syntaxes.

Dry Run Simulation Model:

Many CI tasks can potentially have side effects (modifying files, changing LabVIEW state). A **DryRun** switch is included to allow simulation ⁸⁶. The rationale: - Enable testing and pipeline dry-runs where you want to see what *would* happen (especially destructive actions) without actually performing them. - Allow running a migration (see Migration guide) in a no-op mode to ensure the new actions are wired correctly.

The DryRun is implemented at the adapter level: if DryRun is true, adapters log the intended actions and skip calling the underlying script ⁸⁷ ⁸⁸. This approach was taken because not all underlying scripts had a native “what if” mode. It's simpler to intercept the call at a high level. We decided DryRun would universally: - Prevent any external changes (no g-cli calls, no file writes). - Where feasible, still perform validations. For example, an adapter might parse JSON and ensure required params are present even in DryRun, so that missing params still throw errors immediately rather than falsely indicating success.

One design consideration was whether to incorporate PowerShell's `-WhatIf` infrastructure. We opted for a custom DryRun to have full control, since underlying scripts are not cmdlets and we wanted consistent messaging.

Error Handling and Exit Code Propagation:

Prior to unification, each script handled errors differently (some write to `$LASTEXITCODE`, some throw, some do nothing and rely on external capture). The unified design enforces a consistent error model: - **Strict Mode:** We enable `Set-StrictMode -Version Latest` and `$ErrorActionPreference = 'Stop'` in adapters ⁸⁹. This means any unexpected error becomes a terminating error. - **Try/Catch in Dispatcher:** The dispatcher wraps the adapter invocation in a try/catch ⁹⁰. If an adapter throws (or a PowerShell statement error occurs), we catch it. The dispatcher then: - Writes the error to the error stream (so it's visible in logs) ¹⁶. - Sets an exit code (by default 1 for errors) ¹⁶. We chose 1 for general errors. The exception is if the adapter explicitly threw an exception that corresponds to a known “partial success” scenario with a different code. - **Normalized Codes:** Some actions define special codes: e.g., test failures = 2, missing items = 2, etc., to distinguish from outright errors ⁹¹ ³⁰. The design allows those to propagate. If an underlying script sets `$LASTEXITCODE = 2` (but does not throw), the adapter will not throw but simply return that code. The dispatcher then exits with code 2. This way “expected failure conditions” (like unit tests failing) don't appear as an exception in logs, but still cause a non-zero exit. It's a conscious decision to treat them differently from script errors (like an exception in the code). - **No Silent Failures:** By centralizing error handling, we ensure any error that occurs will lead to a non-zero process exit and visible error message. Inconsistent uses of `Write-Error` vs `throw` across scripts are normalized by the adapter layer (which always ultimately either returns 0 or throws on failure).

Logging and Verbosity:

Consistency in logging was another design goal. Previously, some scripts wrote verbose info, others didn't.

Now: - All adapters use `Write-Information` for key steps, so at `LogLevel=INFO` you see a high-level trace of actions being performed ⁹². - Adapters can use `Write-Verbose` for low-level detail (only shown at `DEBUG` log level). - The dispatcher configures these based on the `-LogLevel` param ¹⁸: - `ERROR`: show only errors. - `WARN`: (we treat similar to `INFO` for now, or could filter). - `INFO`: show information messages (default). - `DEBUG`: show verbose messages as well. - This standardized approach means a user can uniformly increase verbosity for any action. The composite action passes `log_level` input through accordingly ⁹³ ⁹⁴.

Additionally, the dispatcher itself logs an intro message: e.g., `"Invoking action X in directory Y (DryRun? False)"` at `INFO` level, so every run clearly states which action was called and context ²⁰.

Composite Action Wrapper:

The design includes a composite GitHub Action (`abstract-action/action.yml`) to simplify usage in workflows ⁹⁵ ⁷⁸. Rationale: - Users can call the PowerShell script directly in a workflow step, but that's error-prone (need to remember the path, etc.). Wrapping it in a composite action with inputs abstracts those details. - It defines the required inputs (`action_name`, `args_json`) and optional ones (`working_directory`, `dry_run`, `log_level`) with defaults ⁹⁶ ⁹⁷. This provides a self-documenting interface on the Actions marketplace/UI. - The composite uses `shell: pwsh` to invoke the dispatcher script with the given inputs ⁷⁹. It constructs the command string carefully (using an array of parameters to handle quoting) ⁹⁴. - By returning the `$LASTEXITCODE` from the PowerShell call as the step's result (the `exit $exitCode` in dispatcher ensures the step ends with that code), the composite action cause the workflow to fail if the action fails. This is important for CI signaling.

This wrapper means any workflow file just does:

```
- uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: <name>
    args_json: <json>
    # etc.
```

No need to commit or inline the PowerShell script in every repo pipeline.

Backward Compatibility and Migration:

We intentionally did not remove or heavily modify the original scripts. Adapters call them as-is ⁹⁸ ⁹⁹. This yields confidence that behavior remains consistent (same exit codes, same outputs) ⁸⁵. Existing workflows that call scripts directly can continue to do so for now. The unified approach is opt-in.

However, the expectation is to gradually migrate to the unified interface. We provided (or plan to provide) deprecation warnings in the old scripts – e.g., a comment or a runtime warning if they detect they are called directly, suggesting to use `Invoke-OSAAction` instead ¹⁰⁰. Over time, the unified interface will become the primary entypoint, and eventually, direct script calls might be phased out (perhaps by turning those scripts into thin wrappers that call the dispatcher internally, or by removing them in a major release).

The unified design thus allows incremental adoption: one can replace one script call at a time with the composite action, rather than an all-or-nothing switch.

Extensibility:

Adding new actions is easier with this design: - Just drop in a new script (or use existing logic), write a new adapter function, add to registry. No need to design a whole new CLI or error handling for it – follow existing patterns. - The module can be extended without affecting the dispatcher logic (unless new global behavior is needed). - Adapters ensure any new script still conforms to the global contract (so even if a new script author isn't familiar with all the conventions, the adapter can adapt their output).

This was a conscious design choice: prefer a plugin-like approach via the registry. In fact, the registry could even be populated dynamically by scanning for functions, but having it explicit is clearer for maintenance.

Rationale in Context of LabVIEW CI:

LabVIEW build automation has quirks – requires certain LabVIEW versions, tokens, etc. This unified setup centralizes those concerns: - The `SetDevelopmentMode/RevertDevelopmentMode` encapsulate multi-step LabVIEW environment setup/teardown for dev, which used to be error-prone if done manually in workflows. Now it's one action each. - The unified approach also makes cross-platform considerations transparent: For example, if someone tries to run a LabVIEW action on Linux, we could have the dispatcher detect that and throw a clear error (we added a note in the risk mitigations about platform detection ³⁹). Without a unified entry, each script would have needed its own check. - Logging into LabVIEW CLI calls (g-cli) is now uniformly captured. If LabVIEW returns an error, the adapter throws, and we get a consistent error record.

In summary, the design was driven by a need for **consistency, maintainability, and user-friendliness** in orchestrating LabVIEW CI tasks. By introducing a dispatcher and adapter layer, we isolated the older scripts (which are proven but varied) behind a clean interface. This yields a more predictable CI pipeline behavior and simplifies both usage and future development of this toolkit.

Versioning Policy

The **OpenSourceActions** module and composite action follow Semantic Versioning (SemVer) to clearly communicate changes. The version number is stored in the module manifest (`OpenSourceActions.psd1`) as `ModuleVersion`, and tags/releases of the repository use the same version.

Format: **MAJOR.MINOR.PATCH**

- **MAJOR version** – Increases for incompatible changes. This includes:
 - Removal of an action or a breaking rename of an action name.
 - Changes to required input names or types for an existing action (so that workflows using the old inputs would break).
 - Fundamental changes to behavior or output that would likely require users to adjust (e.g., changing exit code conventions, removing DryRun support, etc.).
 - Drops in support (e.g., discontinuing support for a particular LabVIEW version might not trigger MAJOR unless it affects the interface, but if it affects broad usage it could).

When incrementing MAJOR, aim to batch breaking changes and clearly document all of them in release notes.

- **MINOR version** – Increases for added functionality or improvements that are backwards compatible:
 - Adding a new action adapter (new ActionName) is a minor feature addition (existing workflows aren't affected, they just have new options).
 - Adding optional parameters to an existing action (provided existing calls without that param still work with defaults).
 - Changes in underlying behavior that don't require workflow changes but offer new capabilities (e.g., performance improvements, better logging, supporting a new LabVIEW version in an action implicitly).
 - Marking features as deprecated (not removed yet) might accompany a minor if it's just a warning.

MINOR updates should be safe to adopt; they should not break any existing usage, only extend it.

- **PATCH version** – Increases for bug fixes, documentation updates, and other trivial changes that do not affect the interface:
 - Fixing a bug in a script or adapter that doesn't change how you call it, but maybe corrects an exit code or handles an error properly.
 - Minor tweaks to logging messages, typos in output.
 - Internal refactoring that doesn't change outward behavior.
 - Documentation changes (which don't impact runtime).
 - Changes to tests, CI pipeline configs, etc., that don't affect the module's runtime.

PATCH updates should not introduce any new features or change any behaviors that users rely on (other than the bug now being fixed). Ideally, a user could update the patch version without even noticing except the bug is gone.

Initial Development and 0.x:

If the module is pre-1.0, it may not fully follow SemVer guarantees (as SemVer allows breaking changes in 0.x). However, given that this project is already used in CI, we strive to adhere to the spirit of SemVer even in 0.x by minimizing breakage. Once 1.0.0 is released, strict SemVer will be followed.

Updating Version in .psd1

When ready to release a new version: 1. Open `OpenSourceActions.psd1`. Find `ModuleVersion = 'X.Y.Z'`. 2. Update it to the new version number according to the changes made. - If only backwards-compatible features added, increment MINOR, reset PATCH to 0 (e.g., 1.2. **3** -> 1.3.0). - If only bug fixes, increment PATCH (e.g., 1.2.3 -> 1.2.4). - If breaking changes, increment MAJOR, reset MINOR and PATCH to 0 (e.g., 1.4.2 -> 2.0.0). 3. Update `ModuleVersion` and also `GUID` if it's a major change (PowerShell module GUID change is optional, usually keep same GUID for same module unless you want side-by-side installs).

Also check the composite action if it has a version hardcoded (usually it doesn't need a version in yaml itself except for documentation).

Release Notes

Every version change should be accompanied by an update to **ReleaseNotes** (or `CHANGELOG.md` if we maintain one). We often use `Tooling/deployment/release_notes.md` or GitHub Releases to document changes: - List the changes grouped by Added, Changed, Fixed, Removed (especially for major). - For major or minor, highlight if any action names added or any inputs changed. - For patches, describe the bug fixes.

The `generate-release-notes` action can aid in drafting notes from commit history, but you should edit it for clarity and grouping.

If using GitHub Releases, create a release with the tag `vX.Y.Z` and include the notes. If just using a markdown file in repo, update it under a heading for that version.

Examples:

```
## [1.3.0] - 2025-05-10
### Added
- New action: `lint-project` for running VI Analyzer (no breaking changes).

### Changed
- `run-unit-tests` now outputs a summary table (format improved, but no input changes).

### Fixed
- `apply-vipc` now correctly throws on file-not-found (was returning 0 before).

## [1.2.2] - 2025-04-01
### Fixed
- Resolved an issue where `build-lvlibp` would throw a null error if Build_Spec was not found (now provides a clear error message).
```

Keep the tone consistent and mention contributors if applicable.

Maintaining Compatibility

When making changes: - If you think a change might break existing workflows, consider gating it behind a new parameter or clearly call it out in release notes and plan it for a major release. - For deprecations, you can implement a change in a minor (e.g., log a warning that a parameter will be removed in next major) to give users notice.

Version Bump Workflow

Typically: - During development, the version in psd1 stays at the upcoming target (e.g., "1.3.0-dev" or just "1.3.0" if we know next is minor). You could append a suffix like -beta for pre-releases if needed, but PowerShell might not like non-numeric versions in ModuleVersion. - Right before release, ensure all changes since last release are accounted for, then set the final version and tag the release. - After release, if you start working on next version, you can increment in psd1 immediately or keep the same until you have something significant. Just don't forget to bump when releasing.

Composite Action Versioning

The composite GitHub Action shares the versioning. We will likely tag the repository with the same version, so users can use `@v1.3.0` or if we have a moving tag `@v1` for latest minor. Keep in mind: - If you change the composite action inputs or its logic in a breaking way, that might also require a major (but that's unlikely; composite is thin). - We might maintain a major branch tag (like `v1` tag that points to latest 1.x). Typically, create a Git tag for each release.

Document in the release notes how users should reference the action (especially if a major bump happened: e.g., "Update your workflow to use `uses: ...@v2` for version 2.x").

Examples of Changes and Their Impact

- **Example 1:** Add `lint-project` action. This is new functionality, doesn't break anything existing. -> Bump MINOR.
- **Example 2:** Change `build-vi-package` so that it requires a new param that previously was optional (say we decide ReleaseNotesFile should be mandatory – hypothetical). That breaks old usage not providing it. -> Bump MAJOR.
- **Example 3:** Fix `run-unit-tests` parsing to correctly handle special characters in test names. This fix doesn't change how you call it. -> Bump PATCH.
- **Example 4:** Rename `set-development-mode` to `enter-dev-mode`. That's a breaking interface change (action_name different). Even if we keep an alias temporarily, it's effectively breaking. -> Bump MAJOR.
- **Example 5:** Underlying script of `apply-vipc` now supports LabVIEW 2025, but no interface change. -> Bump MINOR (if we consider new LabVIEW support a feature) or maybe PATCH (if it's just an internal update with no user-facing impact besides it now works where it maybe didn't).
- **Example 6:** Remove deprecated action or parameter. -> Bump MAJOR.

We try to batch removals or big changes to avoid frequent major bumps. Minor bumps can be more frequent.

Communication

Communicate version changes via: - Release notes (as discussed). - Possibly README badge or note about latest version. - If a major change, consider also an upgrade/migration note (the docs/migration.md can be updated if needed).

By following this versioning policy, users can pin to a major version for stability or track minor updates for new features, with confidence about the impact of updating. It also helps maintainers plan and merge changes appropriately, grouping breaking changes when necessary.

Migration Guide

So you have been using the individual PowerShell scripts in your CI pipelines and now want to migrate to the unified **Invoke-OSAAction.ps1** (or the composite GitHub Action). This guide will help you transition gradually and safely.

Why Migrate?

- **Simpler Workflows:** One standardized way to call any action, rather than invoking various scripts with different parameters.
- **Better Logging & Errors:** Unified format for logs and error handling means easier debugging.
- **Future Updates:** New features (and fixes) will come to the unified interface first. Some older script calls may be deprecated over time.
- **Cross-Action Coordination:** The unified dispatcher makes it easier to chain actions and manage DryRun for testing flows.

Identify Current Usage

First, find where in your build process you call the old scripts. Common patterns: - In GitHub Actions YAML, steps like:

```
- name: Apply VIPC
  run: pwsh -File actions/apply-vipc/ApplyVIPC.ps1 -MinimumSupportedLVVersion
      2019 -VIPCPATH "My.vipc" ...
```

- Or perhaps in batch/PowerShell scripts outside GitHub.

List all scripts you use (e.g., ApplyVIPC.ps1, Build_lvlibp.ps1, etc.) and note the parameters you pass and any environment context (working directory, etc.).

Mapping Old Calls to New Interface

For each script: - The **ActionName** is typically the folder name or a hyphenated version of the script name. See the table below for common mappings:

Old Script (path)	New ActionName	Notes for ArgsJson
<code>actions/add-token-to-labview/AddTokenToLabVIEW.ps1</code>	<code>add-token-to-labview</code>	Same parameters: MinimumSupportedLVVersion, SupportedBitness, RelativePath.
<code>actions/apply-vipc/ApplyVIPPC.ps1</code>	<code>apply-vipc</code>	Params: MinimumSupportedLVVersion, VIP_LVVersion, SupportedBitness, RelativePath, VIPCPath.
<code>actions/build-lvlibp/Build_lvlibp.ps1</code>	<code>build-lvlibp</code>	Params: MinimumSupportedLVVersion, SupportedBitness, RelativePath, LabVIEW_Project, Build_Spec, Major, Minor, Patch, Build, Commit.
<code>actions/build-vi-package/build_vip.ps1</code>	<code>build-vi-package</code>	Params: SupportedBitness, MinimumSupportedLVVersion, LabVIEWMinorRevision, RelativePath, VIPBPath, Major, Minor, Patch, Build, Commit, DisplayInformationJSON, (optional ReleaseNotesFile).
<code>actions/build/Build.ps1</code>	<code>build</code>	Params: RelativePath, Major, Minor, Patch, Build, Commit, LabVIEWMinorRevision, CompanyName, AuthorName.
<code>actions/close-labview/Close_LabVIEW.ps1</code>	<code>close-labview</code>	Params: MinimumSupportedLVVersion, SupportedBitness.
<code>actions/generate-release-notes/GenerateReleaseNotes.ps1</code>	<code>generate-release-notes</code>	Params: OutputPath (optional).
<code>actions/missing-in-project/Invoke-MissingInProjectCLI.ps1</code>	<code>missing-in-project</code>	Params: LVVersion, Arch, ProjectFile. (The helper script is internal, you only call this one).
<code>actions/modify-vipb-display-info/ModifyVIPBDisplayInfo.ps1</code>	<code>modify-vipb-display-info</code>	Params: SupportedBitness, RelativePath, VIPBPath, MinimumSupportedLVVersion, LabVIEWMinorRevision, Major, Minor, Patch, Build, Commit, DisplayInformationJSON, (optional ReleaseNotesFile).
<code>actions/prepare-labview-source/Prepare_LabVIEW_source.ps1</code>	<code>prepare-labview-source</code>	Params: MinimumSupportedLVVersion, SupportedBitness, RelativePath, LabVIEW_Project, Build_Spec.
<code>actions/rename-file/Rename-file.ps1</code>	<code>rename-file</code>	Params: CurrentFilename, NewFilename.

Old Script (path)	New ActionName	Notes for ArgsJson
<code>actions/restore-setup-lv-source/RestoreSetupLVSource.ps1</code>	<code>restore-setup-lv-source</code>	Params: MinimumSupportedLVVersion, SupportedBitness, RelativePath, LabVIEW_Project, Build_Spec.
<code>actions/revert-development-mode/RevertDevelopmentMode.ps1</code>	<code>revert-development-mode</code>	Params: RelativePath.
<code>actions/run-unit-tests/RunUnitTests.ps1</code>	<code>run-unit-tests</code>	Params: MinimumSupportedLVVersion, SupportedBitness. <i>(Project is assumed from RelativePath or working dir.)</i>
<code>actions/set-development-mode/Set_Development_Mode.ps1</code>	<code>set-development-mode</code>	Params: RelativePath.

Use this as a starting point. The documentation for each action (see `docs/actions/`) details what to put in ArgsJson.

Example Migration

Before (direct script call in GitHub Actions YAML):

```
- name: Build 32-bit Packed Library
  run: |
    pwsh -File actions/build-lvlibp/Build_lvlibp.ps1 `
      -MinimumSupportedLVVersion 2019 `
      -SupportedBitness 32 `
      -RelativePath . `
      -LabVIEW_Project MyProj.lvproj `
      -Build_Spec "My Build" `
      -Major 1 -Minor 0 -Patch 0 -Build 123 -Commit $env:GITHUB_SHA
```

This directly invokes the script with parameters.

After (using composite action):

```
- name: Build 32-bit Packed Library
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: build-lvlibp
    args_json: >
      {
```

```

    "MinimumSupportedLVVersion": "2019",
    "SupportedBitness": "32",
    "RelativePath": ".",
    "LabVIEW_Project": "MyProj.lvproj",
    "Build_Spec": "My Build",
    "Major": 1, "Minor": 0, "Patch": 0,
    "Build": 123, "Commit": "${{ github.sha }}"
  }

```

Key differences: - Using `uses:` with the composite action instead of `run: pwsh -File ...`. - All parameters go into one JSON string (be mindful of quoting and YAML formatting; in the example we used `>` to allow a block style). - Environment variables (like the commit SHA) can be interpolated inside the JSON (as shown).

Working Directory:

If your original script call relied on being in a certain directory (perhaps via `working-directory` in the YAML or a prior `cd` command), ensure to replicate that. The composite action allows a `working_directory` input. For example:

```

- name: Apply VIPC in Project Folder
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:
    action_name: apply-vipc
    working_directory: MyProject # navigate to project folder
    args_json:
      '{"MinimumSupportedLVVersion":"2020","VIP_LVVersion":"2020","SupportedBitness":"64","RelativePath

```

This will push into `MyProject` directory before running, similar to a `working-directory: MyProject` on a run step.

Multiple Script Calls vs Single Action:

Some older processes might call several scripts in sequence. You can often replace them with one or two unified actions: - E.g., previously you might manually do: add token, prepare source, etc. Now `set-development-mode` does all that in one call. - If unsure, migrate step by step – you can replace one script at a time with its corresponding action to verify everything still works.

Incremental Migration with DryRun

A safe approach to migrating critical pipelines: 1. **Parallel Run (DryRun):** Insert the new action in parallel or as an additional step with `dry_run: true` first. For example:

```

- name: (DryRun) Apply VIPC with new action
  uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
  with:

```

```
action_name: apply-vipc
args_json: '{"MinimumSupportedLVVersion":"2019", ... }'
dry_run: true
```

Keep your original script step enabled. The DryRun step will simulate the action and print out what it *would* have done, without changing anything. Compare its log output with what the real step does to ensure it's poised to do the right thing. 2. **Switch to Real Run:** Once satisfied, change `dry_run: true` to false (or remove it, as false is default), and perhaps comment out or remove the old script step. 3. **Monitor Results:** Ensure the pipeline yields the same artifacts/results as before. Because the underlying logic didn't change, things should be equivalent. Check that any files produced have the same content, etc.

If something goes wrong, you can temporarily fall back to the old step (the unified design didn't remove the old scripts, they should still be there). But ideally, fix any issues and continue.

Mapping Arguments Pitfalls

- **Boolean Switches:** In JSON, use `true/false`. E.g., if an action had a switch (none of ours currently is a pure switch except DryRun which is handled separately), you'd use `true`.
- **Paths and Backslashes:** Use forward slashes or escape backslashes as `\\` in JSON strings. Or just ensure YAML quotes properly. Working directory helps avoid long paths.
- **Case Sensitivity:** JSON keys must match exactly the parameter names the adapter expects (usually same as old script). Our adapters typically use same names (sometimes minor casing differences like all parameters in adapter are PascalCase or exact match to script). Check docs/actions file for exact JSON key names. Example: VIP_LVVersion has an underscore and capitals, keep that in JSON key.

Side-by-Side Usage

It's possible to use both old and new concurrently, but it can be confusing. Ideally migrate one pipeline fully. However, you can: - Use the unified actions in new workflows, while leaving legacy ones in old workflows until you get confidence. - Mix within a workflow, though that's usually unnecessary. If you do, remember that the unified actions might set environment (like adding tokens) that persist for the old scripts or vice versa. No conflicts have been seen, but conceptually, e.g., if you call `set-development-mode` (new) and later an old script manually, that's fine – the environment is set up by the new action and old script will run in that environment. Reverse is also fine.

Deprecation of Old Scripts

As of now, the old scripts are still present and callable. Over time: - We might add a warning at the top of old scripts. For example, `AddTokenToLabVIEW.ps1` could output: "WARNING: This script is deprecated. Use `Invoke-OSAction.ps1 -ActionName add-token-to-labview` instead." ¹⁰⁰. - Eventually, in a major release, we could remove them or turn them into thin wrappers that call the unified interface.

Plan your migration such that you aren't caught off guard by such deprecations. This guide and the existence of the unified actions in documentation indicate the direction.

Troubleshooting Tips

- If the unified action fails with an error, compare to how the script was called:
- Did you perhaps miss a required parameter in JSON? The adapter will throw if something mandatory is missing, whereas maybe the old script had a default.
- Check the error message; the dispatcher usually reports what went wrong in a clear way (including underlying script output).
- Use `log_level: DEBUG` on the composite to get verbose logs if needed:

```
with:
  action_name: build-lvlibp
  args_json: '{ ... }'
  log_level: DEBUG
```

This might show more internal info.

- Ensure the `working_directory` is correct. Many actions use `RelativePath` in their params, which typically should be `.` if you want them relative to a certain base. If you notice file-not-found issues, it could be because the action ran in a different directory than expected.
- The unified action runs PowerShell Core. The old scripts ran under whatever shell you invoked (often also Core on GitHub, so that's normally fine).

Before-and-After Workflow Snippet

Before:

```
jobs:
  ci:
    steps:
      - uses: actions/checkout@v3
      - name: Set up Dev Mode
        run: pwsh -File actions/set-development-mode/Set_Development_Mode.ps1 -
RelativePath .
      - name: Run Tests
        run: pwsh -File actions/run-unit-tests/RunUnitTests.ps1 -
MinimumSupportedLVVersion 2020 -SupportedBitness 64
      - name: Build Package 64-bit
        run: pwsh -File actions/build-lvlibp/Build_lvlibp.ps1 -
MinimumSupportedLVVersion 2020 -SupportedBitness 64 -RelativePath . -
LabVIEW_Project My.lvproj -Build_Spec "Build64" -Major 1 -Minor 0 -Patch 0 -
Build $env:BUILDNUM -Commit $env:GITHUB_SHA
      - name: Revert Dev Mode
        run: pwsh -File actions/revert-development-mode/
RevertDevelopmentMode.ps1 -RelativePath .
```

After:

```

jobs:
  ci:
    steps:
      - uses: actions/checkout@v3
      - name: Set up Dev Mode
        uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
        with:
          action_name: set-development-mode
          args_json: '{"RelativePath": "."}'
      - name: Run Tests
        uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
        with:
          action_name: run-unit-tests
          args_json: '{"MinimumSupportedLVVersion": "2020", "SupportedBitness":
"64"}'
      - name: Build Package 64-bit
        uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
        with:
          action_name: build-lvlibp
          args_json: >
            {
              "MinimumSupportedLVVersion": "2020",
              "SupportedBitness": "64",
              "RelativePath": ".",
              "LabVIEW_Project": "My.lvproj",
              "Build_Spec": "Build64",
              "Major": 1, "Minor": 0, "Patch": 0,
              "Build": ${BUILD_NUMBER}, "Commit": "${{ github.sha }}"
            }
      - name: Revert Dev Mode
        uses: LabVIEW-Community-CI-CD/open-source-actions/abstract-action@v1
        with:
          action_name: revert-development-mode
          args_json: '{"RelativePath": "."}'

```

The above refactoring should yield the same results, but now it's cleaner and leverages improvements of the unified system.

Final Tips

- You can always test a command locally by running `Invoke-OSAAction.ps1` on your machine to mimic what the CI would do.
- If confused about JSON structure, create a small PowerShell snippet to build the hashtable and convert to JSON, to ensure formatting. E.g.:

```
$h = @{ MinimumSupportedLVVersion="2020"; SupportedBitness="64";  
RelativePath="." }  
$h | ConvertTo-Json
```

This might help especially for complex nested JSON needed in some actions (e.g., DisplayInformationJSON itself is a JSON string inside the args JSON).

- Keep an eye on release notes of this project; migration might introduce new optional params that you can take advantage of, etc.

Migrating to the unified dispatcher should ultimately save time and reduce errors in your CI process. Take it step by step, use DryRun for confidence, and soon your pipelines will be using the new system seamlessly. Good luck with your migration!

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
91 92 93 94 95 96 97 98 99 100 Unify PowerShell scripts.docx

file:///file-KUsJg84UPHBAMum9FBJaKx